Kelas 11-A

Lapak Buah 11-A

Modul pembelajaran satu minggu: proyek website interaktif dari materi JavaScript, struktur data, DOM, dan ES Module

Mata pelajaran	Pemrograman Web dan Pemrograman Berbasis Teks, Grafis dan Multimedia
Waktu	Senin 19 sampai Kamis 22 Oktober 2026, 5 pertemuan, 10 JP
Teknologi	HTML, CSS native tanpa framework, JavaScript ES Module
Edisi	Modul lengkap untuk guru
Isi	Modul ajar, panduan siswa, rubrik penilaian, lembar kerja siswa, bahan guru



Proyek dan berkas pendukung ada di folder proyek-lapak-buah: contoh-jadi, starter, modul.

Daftar isi

Modul Ajar: Proyek "Lapak Buah 11-A"

1. Gambaran proyek
2. Tujuan pembelajaran
3. Peta materi, fitur, dan file
4. Materi jembatan (hal baru yang perlu diajarkan singkat)
5. Persiapan guru
6. Jadwal satu minggu
7. Rencana per pertemuan
8. Strategi agar siswa memahami, bukan menyalin
9. Diferensiasi dan Jalur Kejar
10. Penilaian
11. Risiko dan cadangan

Panduan Pengerjaan Proyek "Lapak Buah 11-A"

Baca ini dulu

Pertemuan 1: Kerangka halaman dan data

Langkah 1.1 Siapkan dan jalankan
Langkah 1.2 Baca HTML dan buat Peta ID
Langkah 1.3 Header (CSS)
Langkah 1.4 Papan pembuka dan tata letak dua kolom
Langkah 1.5 Katalog dan kartu produk
Langkah 1.6 Data produk

Pertemuan 2: Katalog dinamis

Langkah 2.1 `formatRupiah`
Langkah 2.2 `buatKartu`
Langkah 2.3 `renderKatalog` dan `mulai`

Pertemuan 3: Nota belanja dengan Map

Langkah 3.1 `keranjang.js`
Langkah 3.2 `tambahKeNota` dan daftar item di nota
Langkah 3.3 Pasang event tombol katalog dan uji manual

Pertemuan 4: Diskon, pembayaran, dan kupon

Langkah 4.1 Diskon bertingkat
Langkah 4.2 `hitungRingkasan`
Langkah 4.3 Ringkasan di nota dan kembalian
Langkah 4.4 Tombol Bayar
Langkah 4.5 Kupon (Level 2)

Pertemuan 5: Finishing, uji silang, wawancara kode

Langkah 5.1 Blok A (semua siswa)
Langkah 5.2 Blok B: pilih fitur lanjutan
Langkah 5.3 Uji silang
Langkah 5.4 Wawancara kode dan refleksi

Lampiran A: Contekan CSS yang dipakai

Lampiran B: Materi asal di repositori

Lampiran C: Istilah

Rubrik dan Lembar Penilaian: Proyek "Lapak Buah 11-A"

1. Komponen dan bobot
2. Skala dan cara menghitung
3. Rubrik rinci
4. Contoh pemeriksaan cepat B1 (5 menit per siswa)
5. Lembar penilaian kelas
6. Tindak lanjut hasil
7. Kejujuran akademik

Lembar Kerja Siswa: Proyek "Lapak Buah 11-A"

Pertemuan 1: Senin 19 Oktober 2026
Pertemuan 2: Selasa 20 Oktober 2026 (Jam ke-3)
Pertemuan 3: Selasa 20 Oktober 2026 (Jam ke-6 dan 7)
Pertemuan 4: Rabu 21 Oktober 2026
Pertemuan 5: Kamis 22 Oktober 2026

Bahan Guru: Mini-lesson, Kunci Jawaban, Bank Soal, Troubleshooting

A. Mini-lesson (materi jembatan)
B. Pemanasan dan exit ticket
C. Wawancara kode (Pertemuan 5)
D. Kunci jawaban pertanyaan di panduan siswa
E. Troubleshooting
F. Peta kode `contoh-jadi/`

Modul Ajar: Proyek "Lapak Buah 11-A"

Proyek website interaktif untuk merangkum materi JavaScript, struktur data, DOM, dan ES Module yang sudah diajarkan di kelas 11-A.

Mata pelajaran	Pemrograman Web (PWEB) dan Pemrograman Berbasis Teks, Grafis dan Multimedia (PBTGM)
Kelas	11-A
Waktu	Senin 19 Oktober sampai Kamis 22 Oktober 2026: 5 pertemuan, 10 JP, sekitar 370 menit
Produk akhir	Website "Lapak Buah 11-A": katalog buah, nota belanja, diskon, kupon, pembayaran
Teknologi	HTML, CSS native (tanpa framework CSS), JavaScript ES Module (tanpa library)
Bentuk kerja	Individu. Pair programming pada langkah tertentu
Berkas siswa	`starter/`, `modul/02-panduan-siswa.md`, `modul/04-lembar-kerja-siswa.md`
Berkas guru	`contoh-jadi/`, `modul/01`, `03`, `05`

> Semua jadwal di dokumen ini berasal dari jadwal mengajar yang Bapak/Ibu berikan. "Hari ini" dibaca sebagai Senin 19 Oktober 2026.

---

1. Gambaran proyek

Siswa membangun toko buah kecil yang bisa dipakai langsung:

1. Katalog buah dibuat dari data (`array` berisi `object`), bukan ditulis satu per satu di HTML.
2. Klik "Tambah ke nota" memasukkan buah ke nota belanja. Nota disimpan dalam `Map`.
3. Nota bisa diubah jumlahnya atau dihapus. Subtotal, diskon bertingkat, pembulatan, dan total dihitung otomatis.
4. Pembeli mengisi uang dibayar, lalu halaman menghitung kembalian.
5. Kode program dipecah ke beberapa file `ES Module`.
6. Tampilan memakai CSS native: variabel CSS, grid, flexbox, media query, mode gelap.

Mengapa tema lapak buah

Setiap bagian proyek adalah versi lebih besar dari contoh yang sudah siswa lihat di kelas, sehingga siswa mengenali polanya dan tidak merasa belajar hal asing.

Contoh di kelas	Wujud di proyek
`products` (name, price, category, stok) di `struktur-data/object.js`	Data `produk` di `js/data.js` memakai nama key yang sama
Array `"Apel", "Mangga", "Nanas", "Pisang"` di `manipulasi-array.js`	Nama buah yang dijual
Bonus gaji bertingkat di `challange.js` (>= 5 juta 10%, >= 3 juta 5%, selain itu 0)	Diskon bertingkat di `hitungPersenDiskon`
`sessions` Map dengan key object di `real-case-map.js`	Nota belanja: `Map` dengan key object produk
`multiplier(2)` yang mengembalikan function di `first-class-citizen.js`	`buatKupon(10)` mengembalikan function kupon

Contoh di kelas	Wujud di proyek
`mahasiswa`, `hitungRataRata`, `tentukanPredikat` di `ESModule/config.mjs`	Pola data + fungsi yang dipecah per file
`event.html`: klik tombol mengubah `textContent`, `className`, `src`	Semua interaksi di `main.js`
`sapaByZona()` di `return-value.js`	`sapaByWaktu(jam)` untuk sapaan di halaman

Tiga tingkat target

Tingkat	Isi	Siapa
Wajib	Katalog dari data, nota (Map), diskon bertingkat, pembulatan, bayar dan kembalian, modul ES, CSS responsif, sapaan, papan info, mode gelap	Semua siswa
Level 2	Kupon: `Map` berisi function, `buatKupon` yang mengembalikan function	Target kelas
Bonus	Filter dan urutan katalog, riwayat pesanan (push, shift, pop), tombol ganti warna apel	Siswa yang selesai lebih cepat

---

2. Tujuan pembelajaran

Setelah 5 pertemuan, siswa mampu:

TP	Tujuan	Indikator keberhasilan
TP1	Menyusun halaman responsif dengan HTML semantik dan CSS native	Halaman rapi di lebar 1366 px dan 375 px tanpa scroll mendatar. Warna dan ukuran utama diambil dari `var(--...)`. State "habis" dibuat lewat `className`
TP2	Menyimpan dan mengolah data dengan `array` dan `object`	`data.js` berisi minimal 6 produk valid (tes data lulus). Memakai destructuring minimal 2 tempat
TP3	Memakai `Map` (`set`, `get`, `has`, `delete`) dengan key berupa object	Semua tes keranjang lulus. Siswa bisa menjelaskan mengapa key-nya object
TP4	Menulis function (declaration, expression, arrow), parameter default, return, dan function sebagai nilai	Semua tes helper lulus. Siswa menyebut 3 tempat function dikirim sebagai argumen (`addEventListener`, `forEach`, `setTimeout`, dan lainnya)
TP5	Memakai operator, konversi tipe, dan percabangan/ternary untuk aturan bisnis	Diskon benar di batas 74.999, 75.000, dan 150.000. Input diubah dengan `Number()`. Siswa bisa menjelaskan kenapa `"100000" + 1` menghasilkan `"1000001"`
TP6	Memanipulasi DOM dan event	Katalog dan nota dirender dari data. Memakai event `click`, `input`, `change`. Mengubah `textContent`, `className`, `src`
TP7	Memecah program dengan ES Module	Minimal 4 file module. Memakai named export, default export, dan `import * as`. Siswa bisa menjelaskan kenapa halaman harus dibuka lewat server
TP8	Menguji, mencari kesalahan, dan menjelaskan kode sendiri	Menjalankan `node uji-logika.mjs` dan memperbaiki yang gagal. Ikut uji silang. Lolos satu tugas "ubah-langsung" saat wawancara kode

---

3. Peta materi, fitur, dan file

Kolom "Asal materi" menunjuk berkas di repositori PWEB-11A, supaya siswa bisa membuka kembali contoh asalnya.

Materi yang sudah diajarkan	Asal materi	Wujud di proyek	File
Variabel, `const` dan `let`	`variabel.js`	State halaman, konstanta aturan toko	`main.js`, `helper.js`
Operator aritmetika, `%`, `++`, `--`, `+=`, `-=`	`operator.js`, `pertemuan-4.pdf`	Harga x jumlah, diskon, pembulatan Rp100 dengan `%`, nomor pesanan `++`, stok `-=`	`helper.js`, `main.js`
Operator perbandingan dan logika	`operator.js`, `pertemuan-4.pdf`	`stok > 0`, `subtotal >= batas`, `a || b` (kolom bayar kosong atau nota kosong)	`main.js`, `helper.js`
Ternary	`operator.js`, `arrow-function.js`	Class kartu, label stok, diskon bertingkat	`main.js`, `helper.js`
Konversi implisit dan eksplisit	`konversi-implisit.js`, `konversi-eksplisit.js`	`Number(inputBayar.value)`, `"Rp" + angka`	`main.js`, `helper.js`
Function declaration dan hoisting	`function.js`	`sapaByWaktu`, fungsi `render...`	`helper.js`, `main.js`
Return value	`return-value.js`	Seluruh fungsi helper	`helper.js`, `keranjang.js`
Function expression dan parameter default	`function-experession.js`	`formatRupiah(angka = 0)`	`helper.js`
First-class citizen	`first-class-citizen.js`	`buatKupon`, callback `forEach` / `addEventListener` / `setTimeout` / `sort` / `filter`, default parameter berupa function	`helper.js`, `main.js`
Arrow function	`arrow-function.js`, `challange.js`	Hampir semua helper dan event handler	semua
Array, method array	`array.js`, `manipulasi-array.js`, `array-method.js`	`produk`, `Array.from` untuk salinan, `sort`, `reverse`, `push`, `shift`, `pop`	`data.js`, `main.js`, `riwayat.js`
Array destructuring	`array-destructuring.js`	Pasangan `[item, jumlah]`, tukar gambar apel	`main.js`
Object, destructuring, ubah property	`object.js`	Produk, pesanan, `const { total } = ambilRingkasan()`, `item.stok -= jumlah`	`data.js`, `main.js`
Function di dalam struktur data	`array-object-function.js`	`daftarKupon`: Map berisi function	`helper.js`

Materi yang sudah diajarkan	Asal materi	Wujud di proyek	File
Map	`map.js`, `real-case-map.js`	Nota (key object), daftar kupon	`keranjang.js`, `helper.js`
DOM dan event	`dom/index.html`, `dom/event.html`	Seluruh tampilan interaktif	`main.js`
ES Module	`modularisasi/*`, `ESModule/*`	5 file `.js` saling import	semua `js/`

4. Materi jembatan (hal baru yang perlu diajarkan singkat)

Proyek ini sengaja tidak memakai materi besar baru. Ada delapan mini-lesson, masing-masing 3 sampai 7 menit. Naskah, contoh kode, dan kunci jawabannya ada di `modul/05-bahan-guru.md`.

Kode	Topik	Dipakai di	Mengapa perlu
M1	Server lokal dan `<script type="module">`	Pertemuan 1	Module tidak jalan jika halaman dibuka dengan klik dua kali
M2	Variabel CSS, box model, flex dan grid	Pertemuan 1	Dasar semua CSS proyek
M3	`innerHTML` dan template literal untuk membuat HTML dari data	Pertemuan 2	Cara merender katalog
M4	Pola "render lalu pasang event" dan error `null`	Pertemuan 2	Elemen baru harus ada dulu sebelum diberi event
M5	Map dengan key object dan urutan `Map.forEach` (value, key)	Pertemuan 3	Inti nota belanja
M6	`input.value` selalu string, `Number()`	Pertemuan 4	Kembalian dan pembayaran
M7	Closure: `buatKupon` mengembalikan function	Pertemuan 4 dan 5	Level 2
M8	`sort` dan `filter` menerima function	Pertemuan 5	Bonus

5. Persiapan guru

Sekitar 20 menit sebelum pertemuan pertama.

- [ ] Buka `contoh-jadi/` lewat server dan klik semua fitur sekali, supaya Bapak/Ibu tahu tampilan akhirnya. Cara menjalankan: lihat `README.md` di folder proyek.
- [ ] Jalankan `node uji-logika.mjs` di `contoh-jadi/`. Hasil yang benar: **WAJIB 25 benar, LEVEL 2 5 benar, BONUS 5 benar**.
- [ ] Bagikan folder `starter/` ke siswa (flashdisk, Classroom, atau zip). **Jangan bagikan `contoh-jadi/` sebelum Kamis**, kecuali untuk siswa Jalur Kejar (bagian 9).
- [ ] Pastikan tiap laptop punya VS Code dengan ekstensi Live Server, Node.js (`node -v` menampilkan versi), dan Chrome atau Edge.
- [ ] Siapkan proyektor dengan tiga jendela: browser, VS Code, dan terminal.
- [ ] Cetak atau bagikan `04-lembar-kerja-siswa.md` (satu per siswa).
- [ ] Cetak kartu pertanyaan wawancara dari `05-bahan-guru.md` (bagian C).

- [ ] Atur tempat duduk berpasangan untuk pair programming.
- [ ] Siapkan tabel penilaian dari `03-rubrik-penilaian.md`.

Asumsi ukuran kelas: 20 sampai 24 siswa. Jika berbeda, sesuaikan waktu wawancara di Pertemuan 5.

---

6. Jadwal satu minggu

Pertemuan	Hari dan tanggal	Jam ke	Waktu	Mapel	JP	Menit efektif	Tema
1	Senin 19 Okt 2026	7 sampai 9	13:05 sampai 15:00 (jeda 13:40 sampai 13:45)	PWEB	3	110	Kerangka halaman dan data produk
2	Selasa 20 Okt 2026	3	09:40 sampai 10:20	PWEB	1	40	Katalog dinamis dari data
3	Selasa 20 Okt 2026	6 sampai 7	12:30 sampai 13:40	PBTGM	2	70	Nota belanja dengan Map
4	Rabu 21 Okt 2026	6 sampai 7	12:30 sampai 13:40	PBTGM	2	70	Diskon, pembayaran, kupon
5	Kamis 22 Okt 2026	4 sampai 5	10:20 sampai 11:40	PWEB	2	80	Finishing, uji silang, wawancara kode

Total: 10 JP, 370 menit.

Tentang slot PBTGM (Pertemuan 3 dan 4). Jadwal yang diberikan memuat kelas 11-A pada mapel PBTGM di Selasa dan Rabu, sehingga slot itu dipakai untuk proyek ini. Bagian yang dikerjakan di slot tersebut adalah yang paling dekat dengan PBTGM: logika program berbasis teks (function, `Map`, hitung-hitungan) dan keluaran visual (gambar produk, animasi). Rubrik memisahkan skor logika dan skor tampilan (bagian 10), sehingga dapat dipakai untuk dua mapel jika sekolah memerlukannya. Jika slot PBTGM tidak boleh dipakai untuk proyek ini, kabari saya agar jadwal dipadatkan.

---

7. Rencana per pertemuan

Penanda langkah (`1.3`, `2.2`, dan seterusnya) sama dengan penanda di `starter/` dan `modul/02-panduan-siswa.md`.

Pertemuan 1: Kerangka halaman dan data produk

Senin 19 Oktober 2026, jam ke-7 sampai 9, 13:05 sampai 15:00, PWEB, 110 menit

Tujuan: siswa dapat (a) menjalankan proyek lewat server lokal dan menjelaskan alasannya, (b) menata halaman dengan CSS native, (c) menulis data produk sebagai array of object.

Waktu	Menit	Kegiatan	Langkah
13:05 sampai 13:15	10	**Pembuka.** Tampilkan `contoh-jadi` di proyektor, klik semua fitur. Tanya: "Fitur mana yang memakai materi apa?" Siswa menulis 3 tebakan di lembar kerja. Bagikan peta materi (bagian 3).	-

Waktu	Menit	Kegiatan	Langkah
13:15 sampai 13:25	10	**Setup.** Siswa membuka `starter/` di VS Code dan menjalankan Live Server. Guru lebih dulu membuka `index.html` dengan klik dua kali agar siswa melihat error CORS di Console. Mini-lesson **M1**.	1.1
13:25 sampai 13:40	15	**Baca HTML.** Siswa mengisi "Peta ID" di lembar kerja: 8 id dan perkiraan fungsinya. Guru menjelaskan tag semantik (`header`, `main`, `section`, `aside`, `footer`) dan beda `id` dan `class`. **Checkpoint 1**: server berjalan dan Peta ID terisi.	1.2
13:40 sampai 13:45	5	Pergantian jam pelajaran.	-
13:45 sampai 13:50	5	Mini-lesson **M2**: variabel CSS, box model, flex vs grid.	-
13:50 sampai 14:25	35	**Praktik CSS.** Header (10 menit), papan dan stiker (10), tata letak 2 kolom (10). **Checkpoint 2**: header, papan, dan dua kolom tampil.	1.3, 1.4
14:25 sampai 14:45	20	**Praktik CSS.** Katalog grid dan kartu contoh.	1.5
14:45 sampai 14:55	10	**Data.** Siswa melengkapi `js/data.js` (minimal 6 produk). Jalankan `node uji-logika.mjs`. **Checkpoint 3**: bagian DATA 4 dari 4 lulus.	1.6
14:55 sampai 15:00	5	**Exit ticket** (lihat `05-bahan-guru.md`), simpan pekerjaan.	-

Hasil akhir pertemuan: halaman statis rapi dengan satu kartu contoh, `data.js` berisi 6 produk, tes DATA lulus.

Jika waktu kurang: langkah 1.5 yang belum selesai boleh dilanjutkan di awal Pertemuan 2 (maksimal 5 menit). Siswa yang jauh tertinggal memakai Jalur Kejar (bagian 9).

Catatan guru:

- Langkah 5.1 (CSS responsif) sengaja ditunda ke Pertemuan 5 supaya Pertemuan 1 tidak terlalu padat.
- Bagian nota di CSS sudah disediakan karena memakai teknik lanjutan (conic-gradient, drop-shadow). Siswa tidak menulisnya, tetapi diminta menjelaskan 3 aturan di Pertemuan 5.

Pertemuan 2: Katalog dinamis dari data

Selasa 20 Oktober 2026, jam ke-3, 09:40 sampai 10:20, PWEB, 40 menit

Tujuan: siswa dapat membuat HTML dari data memakai function, `forEach`, dan template literal, lalu menampilkannya lewat DOM.

Waktu	Menit	Kegiatan	Langkah
09:40 sampai 09:45	5	**Pemanasan "tebak output"**: 3 soal tentang destructuring object, `forEach`, template literal (soal dan kunci di `05-bahan-guru.md`).	-
09:45 sampai 09:52	7	Mini-lesson **M3** dan **M4**: membuat daftar siswa dari array object, lalu pola "render, lalu pasang event".	-

Waktu	Menit	Kegiatan	Langkah
09:52 sampai 10:15	23	**Praktik.** `formatRupiah` (5 menit), `buatKartu` (10), `renderKatalog` langkah A sampai C dan `mulai()` (8).	2.1, 2.2, 2.3
10:15 sampai 10:20	5	**Exit ticket**, simpan pekerjaan.	-

Hasil akhir pertemuan: kartu buah tampil dari `data.js`. Harga berformat Rupiah. Produk dengan `stok: 0` tampil dengan stempel "Habis" dan tombol nonaktif.

Checkpoint: ubah `stok` sebuah produk menjadi 0 di `data.js`, muat ulang halaman, kartunya berubah menjadi "Habis" tanpa mengubah kode lain. Inilah bukti bahwa tampilan dibangun dari data.

Jika waktu kurang: guru menunjukkan satu baris contoh `buatKartu` (bagian `className` dengan ternary) di papan tulis. Bagian lain tetap dikerjakan siswa.

Pertemuan 3: Nota belanja dengan Map

Selasa 20 Oktober 2026, jam ke-6 dan 7, 12:30 sampai 13:40, PBTGM, 70 menit

Tujuan: siswa dapat memakai `Map` dengan key object untuk menyimpan nota, dan menampilkan isinya.

Waktu	Menit	Kegiatan	Langkah
12:30 sampai 12:38	8	**Rekap dan mini-lesson M5**: demo `real-case-map.js`, lalu 3 soal tebak (`has` dengan object baru, urutan `Map.forEach`).	-
12:38 sampai 13:08	30	**Praktik `keranjang.js`**: enam fungsi satu per satu, tiap selesai satu jalankan `node uji-logika.mjs`. **Pair programming**: bergantian peran pengetik dan pengamat tiap 2 fungsi.	3.1
13:08 sampai 13:30	22	**Praktik `main.js`**: `tambahKeNota`, `renderNota` bagian daftar item (a sampai e), lalu pasang event tombol "Tambah ke nota".	3.2, 3.3
13:30 sampai 13:37	7	**Uji manual** dengan skenario di panduan siswa (tambah 3 kali, lewat stok, kurangi sampai hilang, hapus).	3.3
13:37 sampai 13:40	3	**Exit ticket.**	-

Hasil akhir pertemuan: klik "Tambah ke nota" memasukkan buah ke nota. Jumlah bisa ditambah, dikurangi, dan dihapus. Tidak bisa melebihi stok. Tes KERANJANG lulus semua.

Jika waktu kurang: utamakan `tambahItem`, `hitungSubtotal`, dan tampilan daftar. Fungsi `kurangiItem` dan tombol `–` boleh diselesaikan di awal Pertemuan 4.

Pertemuan 4: Diskon, pembayaran, dan kupon

Rabu 21 Oktober 2026, jam ke-6 dan 7, 12:30 sampai 13:40, PBTGM, 70 menit

Tujuan: siswa dapat menerapkan operator, ternary, konversi tipe, dan function yang mengembalikan object untuk menghitung diskon, total, dan kembalian.

Waktu	Menit	Kegiatan	Langkah
12:30 sampai 12:35	5	**Rekap.** Tebak output: `"100000" + 1`, `"100000" - 1`, `Number("")`.	-
12:35 sampai 12:42	7	Mini-lesson **M6**: `input.value` selalu string. Ulas ternary bersusun dengan `challange.js`.	-
12:42 sampai 13:12	30	**Praktik helper**: `hitungPersenDiskon`, `hitungDiskon`, `formatPotongan`, `hitungRingkasan`. Hitung 3 kasus dengan tangan dulu, baru menulis kode.	4.1, 4.2
13:12 sampai 13:30	18	**Praktik tampilan**: ringkasan di nota, `renderKembalian`, tombol Bayar.	4.3, 4.4
13:30 sampai 13:37	7	**Level 2 (kupon)**: guru memperagakan `buatKupon` memakai pola `multiplier` (M7). Siswa yang sudah selesai mengerjakan, yang lain lanjut di Pertemuan 5.	4.5
13:37 sampai 13:40	3	**Exit ticket.**	-

Hasil akhir pertemuan (target minimum seluruh kelas): nota menampilkan subtotal, diskon, pembulatan, dan total yang benar. Pembayaran menghitung kembalian. Tes HELPER tingkat WAJIB lulus.

Jika waktu kurang: kupon (langkah 4.5) pindah ke Pertemuan 5. Kupon adalah Level 2, bukan syarat kelulusan.

Pertemuan 5: Finishing, uji silang, wawancara kode

Kamis 22 Oktober 2026, jam ke-4 dan 5, 10:20 sampai 11:40, PWEB, 80 menit

Tujuan: siswa menuntaskan fitur wajib, menguji karya teman, dan menjelaskan kodenya sendiri.

Waktu	Menit	Kegiatan	Langkah
10:20 sampai 10:25	5	**Pembuka**: tampilkan daftar fitur akhir dan aturan wawancara.	-
10:25 sampai 10:45	20	**Blok A (semua siswa)**: CSS responsif (3 media query), tombol tema gelap, sapaan, `renderPapanInfo`.	5.1
10:45 sampai 11:00	15	**Blok B (pilihan sesuai tingkat)**: kupon (Level 2), atau filter dan urutan, atau riwayat pesanan, atau ganti warna apel. Mini-lesson M7 dan M8 bagi yang memilihnya. **Wawancara bergulir dimulai**: guru memanggil siswa yang sudah menyelesaikan Blok A, satu per satu.	4.5, 5.2, 5.4
11:00 sampai 11:10	10	**Uji silang berpasangan** memakai checklist 12 skenario di lembar kerja. Catat bug teman, jangan memperbaikinya.	5.3

Waktu	Menit	Kegiatan	Langkah
11:10 sampai 11:35	25	**Wawancara kode (lanjutan).** Guru melanjutkan memanggil siswa bergantian. Siswa lain memperbaiki bug dari uji silang dan mengisi refleksi.	5.4
11:35 sampai 11:40	5	**Exit ticket** (refleksi), kumpulkan folder proyek, penutup.	-

Hasil akhir pertemuan: proyek lengkap, bug dari uji silang tertangani, wawancara kode selesai, nilai dapat diisi dari rubrik.

Hitungan waktu wawancara: setiap siswa sekitar 2 menit (3 fungsi, 1 pertanyaan konsep, 1 tugas ubah-langsung, dengan kartu pertanyaan sudah disiapkan). Untuk 20 sampai 24 siswa dibutuhkan 40 sampai 48 menit. Waktu itu tersebar sejak 10:45 sampai 11:35 karena wawancara berjalan bergulir saat siswa lain bekerja. Jika masih kurang, wawancarai 5 sampai 6 siswa yang paling cepat selesai sambil berkeliling pada Pertemuan 4. Catat semuanya pada tabel 5.3 di `03-rubrik-penilaian.md`.

Tugas lanjutan (opsional, tanpa nilai): menambahkan fitur kupon sekali pakai, atau menyimpan nota ke `localStorage` (materi baru, bisa jadi bahan pertemuan berikutnya).

---

8. Strategi agar siswa memahami, bukan menyalin

Strategi	Cara kerjanya
Tebak dulu	Sebelum menjalankan kode, siswa menulis tebakan hasilnya di lembar kerja. Selisih antara tebakan dan hasil nyata adalah bahan belajar.
Panduan memakai contoh lain	`02-panduan-siswa.md` mengajarkan pola lewat kasus lain (daftar siswa, absen, ongkir), bukan kode proyek. Siswa harus menerjemahkan sendiri.
Tes otomatis	`node uji-logika.mjs` memberi umpan balik langsung ✅ atau ❌ beserta alasan, sehingga siswa tidak menunggu guru dan tidak bisa menebak-nebak.
Tiga tanya dulu	Urutan bertanya: baca pesan error, tanya teman sebangku, baru tanya guru. Guru membalas dengan pertanyaan balik ("baris mana yang error mengatakan?").
Pair programming	Pengetik dan pengamat bergantian. Pengamat dilarang menyentuh keyboard.
Bantuan bertingkat	Siswa yang macet mendapat Petunjuk 1 (arah), Petunjuk 2 (kerangka), Petunjuk 3 (guru duduk bersama). Jangan langsung memberi jawaban.
Exit ticket harian	3 soal singkat di akhir tiap pertemuan, terhubung dengan hasil hari itu.
Wawancara kode dan ubah-langsung	Di Pertemuan 5 siswa menjelaskan kodenya dan mengubah satu aturan dalam 2 menit (misalnya "ganti batas diskon jadi Rp100.000"). Siswa yang hanya menyalin tidak bisa menyelesaikannya.
Kode jadi disimpan guru	`contoh-jadi/` baru dibuka bersama di Kamis.

---

9. Diferensiasi dan Jalur Kejar

Jalur	Untuk siapa	Isi
A: Wajib	Seluruh siswa	Semua langkah bertanda Wajib
B: Level 2 dan Bonus	Siswa yang selesai lebih cepat	Kupon, filter dan urutan, riwayat, ganti warna. Boleh menambah fitur sendiri setelah dikonsultasikan

Jalur	Untuk siapa	Isi
Kejar	Siswa yang tertinggal lebih dari satu langkah	Setelah ditandatangani guru, boleh menyalin satu berkas dari `contoh-jadi/` (misalnya `css/style.css` agar bisa fokus ke JavaScript). Skor aspek CSS menjadi tingkat 2 paling tinggi. Aspek JavaScript tetap dinilai dari kode yang ia tulis sendiri

Bantuan bertingkat untuk siswa yang macet:

1. **Petunjuk 1** (arah): "Fungsi ini mengembalikan apa? Baca lagi komentar TODO-nya."
2. **Petunjuk 2** (kerangka): guru menulis kerangka tanpa isi, misalnya `const tambahItem = (item) => { /* cek stok */ /* set */ }`.
3. **Petunjuk 3** (duduk bersama): guru dan siswa menelusuri dengan `console.log` langkah demi langkah.

---

10. Penilaian

Ringkasan. Rincian, rubrik, dan lembar penilaian ada di `03-rubrik-penilaian.md`.

Komponen	Bobot	Sumber data
A. Proses dan kemandirian	15	Checkpoint harian, exit ticket, kerja sama
B. Produk	50	Pemeriksaan proyek dan hasil `node uji-logika.mjs`
C. Pemahaman	25	Wawancara kode dan tugas ubah-langsung
D. Kualitas kode dan refleksi	10	Penamaan, komentar, refleksi tertulis

KKM: 75 (angka yang sama dengan batas lulus pada latihan `hitungNilai` di `arrow-function.js`; sesuaikan dengan kebijakan sekolah). Predikat memakai skala yang sama dengan `tentukanPredikat` di `ESModule/config.mjs`: A (90 ke atas), B (80 sampai 89), C (70 sampai 79), D (60 sampai 69), E (di bawah 60).

Pemisahan nilai dua mapel (opsional):

Mapel	Aspek
PBTGM	B2 sampai B5 (logika: operator dan konversi, function, array dan object, Map), ditambah C
PWEB	A, B1, B6 sampai B8 (fungsionalitas, DOM dan event, ES Module, CSS), ditambah D

---

11. Risiko dan cadangan

Risiko	Cadangan
Live Server tidak ada di sebagian laptop	Jalankan `python3 -m http.server 5500` (atau `py -m http.server 5500` di Windows) di folder proyek, buka `http://localhost:5500`. Atau `npx serve`.
Node.js tidak terpasang	Tes otomatis tidak bisa dijalankan. Gunakan Console browser (`console.log`) dan checklist uji manual di panduan siswa.
Laptop kurang	Berpasangan. Satu laptop, dua siswa, bergantian peran tiap 10 menit.
Pertemuan hilang (libur, kegiatan sekolah)	Potong berurutan: bonus, kupon, riwayat. Fitur Wajib tidak dipotong.
Banyak siswa tertinggal di Pertemuan 1	Jalur Kejar untuk CSS. Pertemuan 2 tetap berjalan karena fokusnya JavaScript.

Risiko	Cadangan
Siswa menyalin dari teman	Wawancara kode dan ubah-langsung di Pertemuan 5 menjadi penentu skor aspek Pemahaman.
Siswa memakai AI untuk menjawab	Aturan kelas: AI boleh dipakai untuk bertanya "apa arti error ini", bukan untuk membuat fungsi. Wawancara kode tetap berlaku.
Internet mati	Proyek tidak memakai CDN, font, atau library dari internet, sehingga berjalan offline.

Panduan Pengerjaan Proyek "Lapak Buah 11-A"

Panduan ini membimbing kamu membuat website lapak buah selama satu minggu. Panduannya sengaja **tidak memberikan kode jadi**. Kamu akan diberi arah, contoh dari kasus lain, dan cara memeriksa hasil. Kodenya kamu tulis sendiri, supaya kamu benar-benar paham dan bisa menjelaskannya.

Baca ini dulu

Apa yang kamu buat

Website lapak buah dengan fitur:

- Katalog buah yang dibuat dari data
- Tombol "Tambah ke nota", nota yang bisa diubah jumlahnya
- Diskon otomatis, pembulatan, total, dan kembalian
- Kupon diskon (Level 2)
- Tampilan rapi di laptop dan HP, plus mode gelap

Semua memakai materi yang sudah kamu pelajari: variabel, operator, konversi tipe, function, array, object, Map, DOM, dan ES Module.

Lima aturan main

1. **Tebak dulu, baru jalankan.** Tulis tebakanmu di lembar kerja sebelum menjalankan kode. Salah tebak itu bagus, di situlah kamu belajar.
2. **Ketik sendiri.** Jangan menyalin kode teman atau hasil AI. Contoh di panduan ini memakai kasus lain (siswa, absen, ongkir) supaya kamu menerjemahkannya sendiri ke proyek.
3. **Satu langkah, satu pengujian.** Selesai satu fungsi, langsung jalankan `node uji-logika.mjs` atau lihat di browser. Jangan menumpuk lima fungsi lalu baru mencoba.
4. **Tiga tanya dulu.** Baca pesan error, tanya teman sebangku, baru tanya guru. Guru akan membalas dengan pertanyaan balik.
5. **Simpan dan isi exit ticket** di akhir setiap pertemuan.

Cara membaca panduan

Setiap langkah punya bagian yang sama:

- **Tujuan**: apa yang selesai setelah langkah ini
- **Konsep**: materi yang dipakai, lengkap dengan file asalnya di folder `PWEB-11A`
- **Tebak dulu**: pertanyaan sebelum mencoba (jawab di lembar kerja)
- **Kerjakan**: apa yang harus kamu tulis
- **Cek hasil**: tanda bahwa langkahmu benar
- **Pertanyaan**: jawab di lembar kerja, dengan kata-katamu sendiri
- **Jika macet**: bantuan bertahap, mulai dari petunjuk kecil

Peta file proyek

```
starter/
├── index.html          struktur halaman (SUDAH ADA, kamu baca dan pahami)
├── css/style.css       tampilan (sebagian kamu tulis)
├── uji-logika.mjs      pemeriksa otomatis (tidak diubah)
└── js/
    ├── data.js         DATA produk                          kamu tulis
    ├── helper.js       fungsi hitung-hitungan dan format    kamu tulis
    ├── keranjang.js    isi nota (Map)                       kamu tulis
    ├── riwayat.js      catatan pesanan (bonus)              kamu tulis
    └── main.js         DOM dan event                        kamu tulis
```

Alur data:



`main.js` adalah satu-satunya file yang menyentuh halaman (DOM). File lain hanya berisi data dan logika, sehingga bisa diperiksa lewat terminal tanpa browser.

Alat bantu

Alat	Cara	Gunanya
Live Server	Klik kanan `index.html`, Open with Live Server	Membuka halaman lewat alamat `http://...` (wajib untuk ES Module)
Muat ulang paksa	`Ctrl + Shift + R` (Mac: `Cmd + Shift + R`)	Mengabaikan cache jika perubahan tidak terlihat
DevTools	`F12`, tab **Console** dan **Elements**	Membaca error, memeriksa elemen dan CSS
Terminal VS Code	Menu Terminal, New Terminal	Menjalankan `node uji-logika.mjs`
`console.log`	Tambahkan di kodemu	Mengintip nilai variabel. Hapus jika sudah selesai

Cara membaca pesan error

Pesan di Console	Artinya	Periksa
`blocked by CORS policy ... origin 'null'`	Halaman dibuka dengan klik dua kali (`file://`)	Buka lewat Live Server
`does not provide an export named 'xxx'`	Nama yang di-import tidak ada di file asalnya	Ejaan nama, kata `export` di file asal
`Cannot read properties of null (reading 'addEventListener')`	`querySelector` tidak menemukan elemennya	Ejaan id (`#tambah-1`), dan apakah elemennya sudah dibuat (urutan render)
`xxx is not a function`	Memanggil sesuatu yang bukan function	Ejaan, kurung, apakah sudah di-import
`xxx is not defined`	Variabel belum dibuat atau belum di-import	Ejaan, `import` di bagian atas file
`undefined` tampil di halaman	Function lupa `return`, atau variabel belum diisi	Baris `return`

Pesan di Console	Artinya	Periksa
`NaN` tampil di halaman	Operasi angka dengan nilai yang bukan angka	Apakah ada string yang ikut dihitung (`Number(...)`)

Pertemuan 1: Kerangka halaman dan data

Senin 19 Oktober 2026, 13:05 sampai 15:00. Target: halaman statis rapi dan `data.js` berisi 6 produk.

Langkah 1.1 Siapkan dan jalankan

Tujuan. Halaman starter terbuka lewat alamat `http://...` tanpa error merah di Console.

Konsep. ES Module (`modularisasi/`, `ESModule/`). `<script type="module">` memungkinkan `import` dan `export` di browser.

Tebak dulu. Jika kamu klik dua kali `index.html` (alamatnya `file:///...`), halaman berjalan normal atau tidak? Tulis tebakanmu.

Kerjakan.

1. Salin folder `starter` ke laptopmu, ganti namanya menjadi `lapak-buah-NAMAMU`.
2. Buka folder itu di VS Code (File, Open Folder).
3. Klik kanan `index.html`, pilih **Open with Live Server**.
4. Lihat alamat di browser. Harus diawali `http://127.0.0.1:5500` atau `http://localhost`.
5. Tekan `F12`, buka tab **Console**.
6. Buka terminal VS Code, ketik `node -v` lalu `node uji-logika.mjs`. Banyak tanda ❌ itu **normal** karena kamu belum menulis apa-apa.

Cek hasil. Halaman terbuka (tampilannya masih polos), Console tidak merah, tes menampilkan ringkasan WAJIB / LEVEL 2 / BONUS.

Pertanyaan. Jelaskan dengan kata-katamu: kenapa halaman dengan `<script type="module">` harus dibuka lewat server?

Jika macet.

- Tidak ada Live Server: di terminal ketik `python3 -m http.server 5500` (Windows: `py -m http.server 5500`), lalu buka `http://localhost:5500`.
- `node` tidak dikenali: Node.js belum terpasang, bilang ke guru.

Langkah 1.2 Baca HTML dan buat Peta ID

Tujuan. Kamu tahu elemen mana yang akan dipegang JavaScript.

Konsep. `id` dipakai JavaScript untuk menemukan satu elemen (`querySelector("#nama")`). `class` dipakai CSS untuk menata banyak elemen. Tag semantik (`header`, `main`, `section`, `aside`, `footer`) memberi arti pada bagian halaman. Lihat `dom/index.html`.

Tebak dulu. Buka `js/main.js`, lihat bagian "MEMEGANG ELEMEN". Dari nama variabelnya, tebak 2 elemen yang isinya akan diganti teks dan 2 elemen yang akan diberi event klik.

Kerjakan. Buka `index.html`, temukan elemen yang punya `id`. Isi tabel **Peta ID** di lembar kerja (minimal 8 baris): id, jenis elemen, dan perkiraan apa yang JavaScript lakukan padanya.

Cek hasil. Tabelmu terisi dan tebakanmu sudah dicocokkan dengan `main.js`.

Pertanyaan. Apa yang terjadi di `main.js` kalau id `#tombol-bayar` di HTML kamu ganti menjadi `#tombol-bayar2`?

Langkah 1.3 Header (CSS)

Tujuan. Bilah hijau tua di atas halaman: logo di kiri, dua tombol di kanan, menempel saat halaman di-scroll.

Konsep. Variabel CSS (`--nama`, dipakai `var(--nama)`) dan flexbox. Mini-lesson M2 dari guru.

Contoh dari kasus lain (kartu nilai siswa, bukan header):

```
.kartu-nilai {
  display: flex;                  /* anak-anaknya berjajar dalam satu baris */
  justify-content: space-between; /* sebar: satu ke kiri, satu ke kanan */
  align-items: center;            /* sejajarkan di tengah secara vertikal */
  gap: 12px;                      /* jarak antar anak */
  padding: 12px 16px;
  background: var(--permukaan);   /* nilai dari variabel di :root */
}
```

Tebak dulu. Menurutmu `justify-content` mengatur arah mana, mendatar atau menurun? Dan `align-items`?

Kerjakan. Di `css/style.css`, isi empat blok `LANGKAH 1.3`: `.header`, `.header-isi`, `.logo`, `.header-aksi`. Petunjuk properti ada di komentar tiap blok.

Cek hasil. Header tampil seperti target. Scroll halaman: header tetap di atas.

Pertanyaan. Apa bedanya `justify-content` dan `align-items`? Apa yang terjadi bila `position: sticky` dihapus?

Jika macet.

1. Klik kanan elemen di browser, pilih **Inspect**. Di panel **Styles** kamu bisa mencentang atau mematikan properti untuk melihat efeknya.
2. Pastikan nama class di CSS sama persis dengan di HTML (`.header-isi`, bukan `.headerIsi`).
3. Cek tanda `;` di setiap akhir properti dan `}` di akhir blok.

Langkah 1.4 Papan pembuka dan tata letak dua kolom

Tujuan. Bagian pembuka (judul dan stiker) tertata. Halaman terbagi dua kolom: katalog di kiri, nota di kanan.

Konsep. CSS Grid.

Contoh dari kasus lain:

```
.dua-kolom {
  display: grid;
  grid-template-columns: 1fr 200px; /* kolom kiri fleksibel, kolom kanan 200px */
  gap: 16px;
  align-items: start;               /* tiap kolom setinggi isinya sendiri */
}
```

Tebak dulu. Pada contoh di atas, jika jendela browser dilebarkan, kolom mana yang ikut melebar?

Kerjakan. Isi blok `LANGKAH 1.4`: `.papan`, `.papan-teks p`, `.papan-info`, `.stiker`, `.tata-letak`, `.alat`, dan `select, input`. Stiker "Diskon 5%" yang ada di HTML adalah contoh agar kamu bisa melihat hasilnya.

Cek hasil. Dua kolom terlihat. Nota (kertas krem) ada di kolom kanan. Kecilkan jendela: bagian kirinya menyempit, bukan mendorong nota keluar layar.

Pertanyaan. Komentar di `.tata-letak` meminta `minmax(0, 1fr)`, bukan `1fr` saja. Cari tahu apa bedanya, lalu tulis jawabanmu.

Langkah 1.5 Katalog dan kartu produk

Tujuan. Kartu produk rapi: foto berlatar warna, label harga menempel di foto, nama, info, dan tombol.

Konsep. `repeat(auto-fill, minmax(...))`, `position: relative` dan `absolute`, variabel CSS yang diisi dari HTML (`style="--warna: ..."`).

Contoh dari kasus lain:

```
.galeri {
  display: grid;
  /* "sebanyak mungkin kolom selebar minimal 150px, sisanya dibagi rata" */
  grid-template-columns: repeat(auto-fill, minmax(150px, 1fr));
  gap: 12px;
}
```

Tebak dulu. Jika `.galeri` berada di layar selebar 600 px, berapa kolom yang muncul? Kalau layar 300 px?

Kerjakan. Isi blok yang masih `TODO` di bagian `LANGKAH 1.5`: `.katalog`, `.kartu`, `.kartu-foto`, `.kartu-foto img`, `.kartu-isi`, `.kartu-info`, `.kartu-isi .tombol`, `.kartu.habis .kartu-foto img`. Label harga dan stempel "Habis" sudah disediakan guru, baca saja.

Cek hasil.

- Kartu apel tampil dengan latar merah muda, label harga di sudut kiri bawah foto.
- Ubah `class="kartu"` di HTML menjadi `class="kartu habis"`: gambar menjadi abu-abu dan muncul stempel "Habis". Kembalikan setelah dicoba.
- Lebarkan dan kecilkan jendela. Jumlah kolom berubah sendiri.

Pertanyaan. Kenapa `.kartu-foto` harus `position: relative`? (Petunjuk: lihat `.harga-tag`.) Dari mana warna latar foto diambil?

Langkah 1.6 Data produk

Tujuan. `js/data.js` berisi minimal 6 produk valid.

Konsep. Array berisi object, named export. Lihat `struktur-data/array.js`, `struktur-data/object.js`, dan `ESModule/config.mjs` (array `mahasiswa`).

Contoh dari kasus lain (`ESModule/config.mjs`):

```
export const mahasiswa = [
  { nama: "Maulana", nilai: 90 },
  { nama: "Dimas", nilai: 80 },
];
```

Tebak dulu. Jika harga ditulis `"18.000"` (string, pakai titik), apa hasil `"18.000" * 2`? Coba di Console browser.

Kerjakan. Lengkapi `data.js` mengikuti aturan di komentarnya. Pastikan ada minimal satu produk dengan `stok: 0`.

Agar hasil hitunganmu di Pertemuan 3 dan 4 sama dengan contoh di panduan, pakai data berikut. Kamu boleh mengubahnya setelah semuanya selesai.

id	name	price	category	stok	gambar	warna
1	Apel	32000	Impor	12	`apel-merah.svg` (gambarAlt: `apel-hijau.svg`)	`#ffe1de`
2	Pisang Ambon	18000	Lokal	20	`pisang.svg`	`#fff1b8`
3	Mangga Harum Manis	25000	Lokal	4	`mangga.svg`	`#ffe7bd`
4	Nanas Madu	15000	Lokal	0	`nanas.svg`	`#f4f0c4`
5	Jeruk Medan	22000	Lokal	15	`jeruk.svg`	`#ffddc2`
6	Anggur Ungu	60000	Impor	6	`anggur.svg`	`#e9ddff`

Cek hasil. Di terminal jalankan `node uji-logika.mjs`. Bagian **DATA** harus ✅ semua (4 tes).

Pertanyaan. Kenapa `id` harus unik? Di bagian mana `id` produk akan dipakai nanti? (Petunjuk: lihat komentar `buatKartu` di `main.js`.)

Exit ticket Pertemuan 1: kerjakan di lembar kerja, lalu simpan pekerjaanmu.

Pertemuan 2: Katalog dinamis

Selasa 20 Oktober 2026, 09:40 sampai 10:20. Target: kartu buah tampil dari `data.js`.

Langkah 2.1 `formatRupiah`

Tujuan. `formatRupiah(32000)` menghasilkan `"Rp32.000"`, dan `formatRupiah()` menghasilkan `"Rp0"`.

Konsep. Function expression dan parameter default: `function-experession.js`, `function.js`. Operator `+` sebagai penggabung string: `pertemuan-4.pdf`.

Contoh dari kasus lain:

```
const formatPersen = function (angka = 0) {
  return angka + "%";
};
formatPersen(15); // "15%"
formatPersen();   // "0%"
```

Tebak dulu. Ketik `(32000).toLocaleString("id-ID")` di Console browser. Apa hasilnya dan apa tipe datanya (`typeof`)?

Kerjakan. Tulis `formatRupiah` di `js/helper.js` sebagai **function expression** dengan parameter default.

Cek hasil. `node uji-logika.mjs`: tes `formatRupiah` ✅.

Pertanyaan. Apa bedanya function expression dan function declaration saat dipanggil sebelum barisnya? (Petunjuk: `function-experession.js` baris atas.)

Langkah 2.2 `buatKartu`

Tujuan. `buatKartu(item)` menerima satu object produk dan mengembalikan string HTML satu kartu.

Konsep. Object destructuring (`object.js`), template literal, ternary. Mini-lesson M3.

Contoh dari kasus lain:

```
const buatBarisSiswa = (siswa) => {
  const { nama, nilai } = siswa;
  const lulus = nilai >= 75;
  return `<li class="${lulus ? "lulus" : "remedial"}">${nama}: ${nilai}</li>`;
};
buatBarisSiswa({ nama: "Dimas", nilai: 80 });
// '<li class="lulus">Dimas: 80</li>'
```

Tebak dulu. Apa tipe data hasil `buatBarisSiswa(...)`: object, array, atau string?

Kerjakan. Di `js/main.js`, isi `buatKartu`. Baca daftar "yang harus berubah mengikuti data" di komentarnya. Struktur HTML-nya sama dengan kartu contoh di `index.html`.

Cek hasil. Sementara tambahkan `console.log(buatKartu(produk[0]))` di bawah fungsi itu, simpan, dan lihat hasilnya di Console. HTML-nya harus mirip kartu contoh. Hapus `console.log` sesudahnya.

Pertanyaan. Kenapa `buatKartu` mengembalikan string dan tidak langsung mengisi halaman? (Petunjuk: bisa dipakai untuk setiap produk, dan mudah diuji.)

Langkah 2.3 `renderKatalog` dan `mulai`

Tujuan. Semua produk dari `data.js` tampil sebagai kartu.

Konsep. `forEach`, `innerHTML`, pola "render, lalu pasang event". Mini-lesson M3 dan M4.

Contoh dari kasus lain:

```
const daftarSiswa = [
  { nama: "Rizky", nilai: 80 },
  { nama: "Dimas", nilai: 90 },
];
const daftar = document.querySelector("#daftar");

let html = "";                        // wadah teks, mulai dari kosong
daftarSiswa.forEach((siswa) => {
  html += buatBarisSiswa(siswa);      // tambahkan satu baris tiap putaran
});
daftar.innerHTML = html;              // baru ditulis ke halaman, satu kali
```

Tebak dulu. Jika baris `let html = "";` diganti `let html;` (tanpa nilai), apa isi `html` di akhir? (Coba, hasilnya mengejutkan.)

Kerjakan. Isi `renderKatalog` untuk **Langkah A sampai C** saja. Langkah D (pasang event tombol) dikerjakan di Pertemuan 3. Lalu isi `mulai()` agar memanggil `renderKatalog()` dan `renderNota()`.

Cek hasil.

- Kartu contoh statis hilang, diganti 6 kartu dari `data.js`.
- Ubah `stok` sebuah produk menjadi `0`, muat ulang: kartunya otomatis menjadi "Habis".
- Teks "6 jenis buah ditampilkan" muncul.

Pertanyaan. Kenapa event tidak bisa dipasang pada tombol di dalam kartu sebelum `katalog.innerHTML = html`? Apa pesan error yang akan muncul?

Exit ticket Pertemuan 2: kerjakan di lembar kerja.

Pertemuan 3: Nota belanja dengan Map

Selasa 20 Oktober 2026, 12:30 sampai 13:40. Target: tombol "Tambah ke nota" berfungsi.

Langkah 3.1 `keranjang.js`

Tujuan. Enam fungsi nota lulus semua tes KERANJANG.

Konsep. Map: `set`, `get`, `has`, `delete`, `clear`, `forEach`. Key boleh berupa object. Lihat `struktur-data/map.js` dan `struktur-data/real-case-map.js`. Mini-lesson M5.

Contoh dari kasus lain (mencatat kehadiran):

```
const absen = new Map();
const rizky = { nama: "Rizky" };

absen.set(rizky, 1);                          // key = object, value = jumlah hadir
absen.has(rizky);                             // true
absen.get(rizky);                             // 1
absen.set(rizky, absen.get(rizky) + 1);       // naik jadi 2
absen.delete(rizky);                          // dibuang
absen.forEach((jumlah, siswa) => { /* ... */ }); // urutan parameter: (value, key)
```

Tebak dulu.

```
const a = { id: 1 };
const b = { id: 1 };
const peta = new Map();
peta.set(a, "x");
console.log(peta.has(a), peta.has(b), peta.size);
```

Tulis tiga hasilnya sebelum menjalankan di Console.

Kerjakan (berpasangan). Kerjakan satu fungsi, jalankan `node uji-logika.mjs`, lihat ✅, baru lanjut. Pengetik dan pengamat bertukar tiap 2 fungsi. Urutan: `tambahItem`, `kurangiItem`, `hapusItem`, `kosongkanKeranjang`, `hitungSubtotal`, `hitungJumlahItem`.

Cek hasil. Bagian **KERANJANG** di tes ✅ semua (9 tes).

Pertanyaan.

1. Kenapa `peta.has(b)` bernilai `false` padahal isi `a` dan `b` sama?
2. Apa urutan parameter callback `Map.forEach`? Apa bedanya dengan `Array.forEach`?
3. Apa yang terjadi bila `tambahItem` tidak mengecek stok?

Jika macet.

1. Tulis dulu dengan kalimat biasa: "Kalau item belum ada di nota, jumlah sekarang 0. Kalau sudah ada, ambil jumlahnya." Lalu terjemahkan ke `has` dan `get`.
2. Baca pesan ❌ di terminal. Pesannya menyebut nilai yang diharapkan dan nilai yang kamu hasilkan.
3. Pasang `console.log(keranjang)` di dalam fungsimu untuk melihat isi Map.

Langkah 3.2 `tambahKeNota` dan daftar item di nota

Tujuan. `renderNota` menampilkan daftar item dari Map.

Konsep. Alur data satu arah:

```
klik tombol
   │
   ▼
tambahKeNota(item) ──► tambahItem(item)      (keranjang.js, ubah data)
   │
   ▼
renderNota() ──► keranjang.forEach(...) ──► string HTML ──► daftarNota.innerHTML
   │
   └──► pasang event untuk tombol −, +, Hapus yang baru dibuat
```

Setiap kali data berubah, HTML dibangun ulang dari data. Kamu tidak mengutak-atik elemen satu per satu.

Tebak dulu. Pada baris nota, apa yang harus unik untuk setiap produk supaya `querySelector` bisa menemukan tombol yang benar? Dari mana nilainya diambil?

Kerjakan. Di `main.js`, isi `tambahKeNota`, lalu bagian **Langkah 3.2** di `renderNota` (poin a sampai e). Satu baris nota memakai elemen berikut (nama class sudah ada di CSS):

Elemen	Isi
`img`	Foto kecil produk
`div.baris-info`	`strong` nama produk dan `span` "Rp... x N kg"
`div.stepper`	`button` kurangi, `output` jumlah, `button` tambah
`span.baris-harga`	Harga x jumlah
`button.hapus`	Teks "Hapus"

Beri tombol id yang unik: `kurang-${item.id}`, `tambah-nota-${item.id}`, `hapus-${item.id}`. Tombol yang hanya berisi simbol (`−`, `+`) perlu `aria-label` agar terbaca oleh pembaca layar.

Cek hasil. Pada tahap ini tombol di katalog belum berfungsi. Uji sementara: di `main.js`, tepat di atas baris `mulai();` tulis `tambahItem(produk[0]);`. Pastikan `mulai()` memanggil `renderNota()`. Nota harus menampilkan Apel 1 kg. Hapus baris uji itu sesudahnya.

Pertanyaan. Kenapa parameter `keranjang.forEach` ditulis `(jumlah, item)` dan bukan `(item, jumlah)`?

Langkah 3.3 Pasang event tombol katalog dan uji manual

Tujuan. Tombol "Tambah ke nota" di setiap kartu berfungsi.

Kerjakan. Kembali ke `renderKatalog`, kerjakan **Langkah D** (pasang event klik tiap tombol). Jalankan skenario berikut:

No	Skenario	Hasil yang benar
1	Klik "Tambah ke nota" pada Apel 3 kali	Nota: Apel 3 kg. Angka di tombol Nota menjadi 3
2	Klik tombol `−` pada Apel	Jadi 2 kg

No	Skenario	Hasil yang benar
3	Klik "Tambah ke nota" pada Mangga (stok 4) sebanyak 5 kali	Jumlah berhenti di 4 kg dan muncul pesan stok tinggal 4 kg. Tombol `+` di nota ikut nonaktif
4	Klik `−` pada item yang tinggal 1 kg	Item hilang dari nota
5	Klik "Hapus" pada item dengan 3 kg	Item hilang sekaligus
6	Cek kartu produk dengan stok 0	Tombol nonaktif, tidak bisa diklik

Exit ticket Pertemuan 3: kerjakan di lembar kerja.

Pertemuan 4: Diskon, pembayaran, dan kupon

Rabu 21 Oktober 2026, 12:30 sampai 13:40. Target: total dan kembalian benar.

Langkah 4.1 Diskon bertingkat

Tujuan. Tiga fungsi lulus tes: `formatPotongan`, `hitungPersenDiskon`, `hitungDiskon`.

Konsep. Arrow function, ternary bersusun, operator perbandingan. Lihat bonus bertingkat di `challange.js` (gaji >= 5.000.000 dapat 10%).

Contoh dari kasus lain (ongkir bertingkat):

```
const hitungOngkir = (jarakKm) => (jarakKm >= 10 ? 15000 : jarakKm >= 5 ? 8000 : 0);
hitungOngkir(12); // 15000
hitungOngkir(5);  // 8000
hitungOngkir(2);  // 0
```

Tebak dulu (sebelum mengetik kode). Hitung dengan tangan dan isi tabel di lembar kerja:

Subtotal	Persen diskon	Diskon (Rp)
60.000	?	?
74.999	?	?
75.000	?	?
150.000	?	?

Kerjakan. Di `helper.js` isi `formatPotongan`, `hitungPersenDiskon`, dan `hitungDiskon`. Gunakan konstanta `BATAS_DISKON_SEDANG` dan `BATAS_DISKON_TINGGI`, jangan menulis angka 75000 dan 150000 langsung.

Cek hasil. Tes `formatPotongan`, `hitungPersenDiskon`, `hitungDiskon` ✅, dan hasilnya sama dengan tabel tebakanmu.

Pertanyaan. Rumus diskon di proyek ini ditulis `subtotal * persen / 100` (kalikan dulu, baru bagi). Coba `3 * 5 / 100` dan `3 * (5 / 100)` di Console. Apa bedanya? Kenapa urutan pertama lebih aman untuk uang?

Langkah 4.2 `hitungRingkasan`

Tujuan. Satu fungsi yang mengembalikan seluruh angka nota dalam bentuk object.

Konsep. Return object, parameter default berupa function, operator modulus (`%`). Lihat `pertemuan-4.pdf` (modulus) dan `function.js` (parameter default).

Tebak dulu.

1. Berapa `73150 % 100`?
2. Apa hasil `((a, f = () => 0) => f(a))(5)`? Tulis tebakanmu, lalu cek di Console.

Hitung dengan tangan dulu. Lengkapi tabel ini di lembar kerja:

Subtotal	Kupon	Diskon	Potongan kupon	Sebelum pembulatan	Pembulatan	Total
100.000	tidak ada	?	?	?	?	?
77.000	tidak ada	?	?	?	?	?
80.000	10% (Level 2)	?	?	?	?	?

Kerjakan. Tulis `hitungRingkasan(subtotal, fungsiKupon)` sesuai spesifikasi di komentarnya. Parameter kedua harus punya nilai default berupa function yang selalu mengembalikan 0.

Cek hasil. Tes `hitungRingkasan` ✅ dan sama dengan tabelmu. (Tes yang memakai kupon baru lulus setelah Langkah 4.5.)

Pertanyaan. Untuk apa pembulatan ke bawah kelipatan Rp100 di toko sungguhan? Kenapa memakai `%` dan bukan pembagian biasa?

Langkah 4.3 Ringkasan di nota dan kembalian

Tujuan. Nota menampilkan subtotal, diskon, kupon, pembulatan, total. Kolom bayar menghitung kembalian.

Konsep. Object destructuring dari hasil function, konversi eksplisit: `konversi-implisit.js`, `konversi-eksplisit.js`. Mini-lesson M6.

Tebak dulu. Kolom bayar berisi `100000`. Tulis hasil:

```
inputBayar.value + 1000      // ?
Number(inputBayar.value) + 1000  // ?
inputBayar.value - 1000      // ?
Number("")                   // ?
```

Kerjakan.

1. Di `renderNota`, kerjakan poin f sampai i (ringkasan harga).
2. Isi `renderKembalian`.
3. Pasang event `input` pada `inputBayar`. Perhatikan: `renderKembalian` dikirim **tanpa tanda kurung**.

Cek hasil.

- Tambahkan Apel 3 kg dan Anggur 1 kg (subtotal Rp156.000): diskon 10%, total Rp140.400.
- Isi kolom bayar `100000`: tertulis "Uang kurang Rp40.400" berwarna merah. Isi `150000`: "Kembalian Rp9.600" berwarna hijau.
- Kosongkan kolom bayar: pesan hilang.

Pertanyaan.

1. Kenapa `inputBayar.value` selalu string walaupun `type="number"`?
2. Apa bedanya `addEventListener("input", renderKembalian)` dengan `addEventListener("input", renderKembalian())`?

Langkah 4.4 Tombol Bayar

Tujuan. Klik Bayar memeriksa uang, mengurangi stok, dan mengosongkan nota.

Kerjakan. Isi event `tombolBayar` mengikuti sembilan langkah di komentarnya. Pada langkah 6, baris `riwayat.catatPesanan(pesanan)` boleh dikosongkan dulu karena bonus, tetapi `nomorPesanan++` tetap dikerjakan.

```
klik Bayar
   ├─ nota kosong?        ──► toast, berhenti
   ├─ uang kurang?        ──► toast, berhenti
   └─ cukup ──► buat object pesanan ──► kurangi stok ──► reset nota ──► render ulang ──► toast
```

Cek hasil.

Skenario	Hasil yang benar
Klik Bayar saat nota kosong	Pesan nota masih kosong
Bayar kurang dari total	Pesan uang kurang, nota tetap
Bayar pas atau lebih	Pesan pesanan dibayar dan kembalian. Nota kosong. Stok di kartu berkurang
Beli seluruh stok Mangga lalu bayar	Kartu Mangga berubah menjadi "Habis"

Pertanyaan. Kenapa stok dikurangi **setelah** pembayaran, bukan saat item dimasukkan ke nota?

Langkah 4.5 Kupon (Level 2)

Tujuan. Kode kupon memotong harga. Tes **LEVEL 2** lulus semua (5 tes).

Konsep. Function yang mengembalikan function (`first-class-citizen.js`, bagian `multiplier`), Map yang berisi function (`struktur-data/map.js` dan `array-object-function.js`). Mini-lesson M7.

Contoh dari kasus lain (pajak):

```
const buatPajak = (persen) => (harga) => (harga * persen) / 100;

const ppn11 = buatPajak(11);   // ppn11 sekarang adalah function
ppn11(100000);                 // 11000

const daftarPajak = new Map([["PPN11", buatPajak(11)], ["PPN5", buatPajak(5)]]);
daftarPajak.get("PPN11")(100000); // 11000  (ambil function dari Map, langsung panggil)
```

Tebak dulu. Pada `buatPajak(11)`, di mana angka `11` "disimpan" sehingga masih diingat saat `ppn11(100000)` dipanggil jauh sesudahnya?

Kerjakan.

1. Tulis `buatKupon(persen)` di `helper.js`.
2. Isi `daftarKupon` dengan tiga kupon: `HEMAT5` (5%), `HEMAT10` (10%), `KELAS11A` (15%).
3. Di `main.js`, isi event `tombolKupon`.

Cek hasil. Tes LEVEL 2 ✅. Di halaman: kupon `kelas11a` (huruf kecil) tetap diterima, kupon `ngawur` menampilkan pesan merah, nota ikut berubah.

Pertanyaan.

1. Kenapa `daftarKupon.get("HEMAT10")(200000)` bisa langsung dipanggil seperti function?
2. Jika ingin menambah kupon `BUAH20` (20%), file mana yang diubah dan berapa baris?

Exit ticket Pertemuan 4: kerjakan di lembar kerja.

Pertemuan 5: Finishing, uji silang, wawancara kode

Kamis 22 Oktober 2026, 10:20 sampai 11:40.

Langkah 5.1 Blok A (semua siswa)

Tujuan. Halaman responsif, mode gelap, sapaan, dan papan info berfungsi.

Konsep. Media query (`@media`), `className` dan `textContent` (`dom/event.html`), function declaration (`function.js`, `return-value.js`).

Kerjakan.

1. **CSS responsif.** Isi tiga bagian `LANGKAH 5.1` di akhir `style.css`. Uji dengan DevTools: tekan `Ctrl + Shift + M` (mode perangkat), pilih lebar 375 px dan 1366 px.
2. **Mode gelap.** Isi event `tombolTema` di `main.js`. CSS gelap sudah disediakan (`body.gelap`).
3. **Sapaan.** Tulis `sapaByWaktu(jam)` di `helper.js` sebagai function declaration, lalu pakai di `mulai()`.
4. **Papan info.** Isi `renderPapanInfo`. Angka persen dan batas rupiah diambil dari konstanta dan fungsi di `helper.js`, bukan ditulis manual.

Cek hasil.

Cek	Hasil yang benar
Lebar 375 px	Tidak ada scroll mendatar. Katalog 2 kolom. Nota di bawah katalog
Lebar 1366 px	Dua kolom besar. Nota di kanan
Klik "Mode gelap"	Warna berubah, teks tombol menjadi "Mode terang". Klik lagi kembali terang
Sapaan	Sesuai jam laptop (pagi, siang, sore, malam)
Ubah `BATAS_DISKON_SEDANG` jadi `100000`	Papan info ikut berubah tanpa mengubah `main.js`

Pertanyaan. Kenapa mode gelap cukup menimpa nilai variabel CSS dan tidak perlu menulis ulang semua aturan?

Langkah 5.2 Blok B: pilih fitur lanjutan

Pilih sesuai kemampuanmu. Minimal satu bila Blok A dan Langkah 4.5 sudah selesai.

B1. Filter dan urutan (Mini-lesson M8)

Konsep. `sort` dan `filter` menerima function. `Array.from` untuk menyalin array. `array-method.js`.

Tebak dulu.

```
const angka = [10, 9, 100, 1];
angka.sort();                    // ?
angka.sort((a, b) => a - b);     // ?
```

Kerjakan. Di `renderKatalog` (bagian BONUS), tambahkan filter kategori dan urutan harga. Pasang event `change` untuk `pilihKategori` dan `pilihUrutan`.

Cek hasil. Kategori "Impor" hanya menampilkan 2 buah (jika datamu sama dengan contoh). Urutan "Harga termurah" benar. Array `produk` asli tidak berubah urutannya.

Pertanyaan. Kenapa harus `Array.from(produk)` dulu sebelum `sort`?

B2. Riwayat pesanan

Konsep. `push`, `shift`, `pop` (`manipulasi-array.js`), `Array.from` dan `reverse` (`array.js`, `array-method.js`), array destructuring di parameter.

Kerjakan. Tulis tiga fungsi di `riwayat.js`, isi `renderRiwayat`, event `tombolBatal`, dan aktifkan langkah riwayat di event Bayar.

Cek hasil. Tes **BONUS** ✅ (5 tes). Riwayat maksimal 5 pesanan, yang paling baru di atas. Batalkan pesanan terakhir mengembalikan stok.

Pertanyaan. `reverse()` mengubah array aslinya. Bagaimana kamu menampilkan urutan terbaru di atas tanpa merusak urutan asli?

B3. Ganti warna apel

Konsep. Array destructuring untuk menukar dua nilai: `[a, b] = [b, a]`. Ubah `img.src` (`dom/event.html`).

Kerjakan. Pada kartu yang punya `gambarAlt`, pasang event pada tombol "Ganti warna" untuk menukar `gambar` dan `gambarAlt`, lalu mengganti `src` gambar.

Langkah 5.3 Uji silang

Tujuan. Menemukan bug di karya temanmu tanpa memperbaikinya.

Kerjakan. Tukar laptop dengan teman. Jalankan 12 skenario di lembar kerja (bagian "Uji silang"). Catat setiap kegagalan: skenario nomor berapa, apa yang terjadi, apa yang seharusnya. Kembalikan laptop dan perbaiki bug dari temanmu pada karyamu sendiri.

Langkah 5.4 Wawancara kode dan refleksi

Tujuan. Menunjukkan bahwa kamu memahami kodemu sendiri.

Persiapan. Pilih tiga fungsi yang kamu tulis sendiri. Untuk masing-masing, latih menjelaskan dengan pola:

> "Fungsi ini menerima ... , melakukan ... , mengembalikan ... . Dipakai di ... ."

Saat wawancara, guru akan:

1. Menunjuk satu fungsi dan memintamu menjelaskan.
2. Mengajukan satu pertanyaan konsep dari materi.
3. Meminta satu **perubahan langsung** (sekitar 2 menit), misalnya:
   - ganti batas diskon 5% menjadi Rp100.000
   - tambahkan kupon `BUAH20` (20%)

- batasi pembelian maksimal 5 kg per jenis
- ubah warna utama tombol menjadi biru

Siswa yang menulis sendiri kodenya tahu persis file mana yang diubah.

Refleksi (tulis di lembar kerja, 3 sampai 5 kalimat): bagian apa yang paling kamu pahami, bagian apa yang masih membingungkan, dan apa rencanamu mempelajarinya.

Lampiran A: Contekan CSS yang dipakai

Kebutuhan	Properti
Menyusun anak dalam satu baris atau kolom	`display: flex`, `flex-direction`, `justify-content`, `align-items`, `gap`, `flex-wrap`
Menyusun dalam kolom dan baris	`display: grid`, `grid-template-columns`, `gap`
Kolom otomatis menyesuaikan layar	`repeat(auto-fill, minmax(210px, 1fr))`
Jarak dalam dan luar	`padding`, `margin`, `margin-inline: auto` (tengahkan)
Garis tepi dan sudut	`border`, `border-radius`, `overflow: hidden`
Warna dari variabel	`color`, `background`, `var(--nama)`
Posisi	`position: relative`, `absolute`, `sticky`, `top`, `left`, `bottom`
Gambar	`width`, `max-width: 100%`, `filter: grayscale(1)`, `opacity`
Aturan khusus layar	`@media (max-width: 900px) { ... }`
Keadaan elemen	`:hover`, `:disabled`, `:focus-visible`

Lampiran B: Materi asal di repositori

Topik	Buka berkas
Operator, ternary	`operator.js`, `pertemuan-4.pdf`
Konversi tipe	`konversi-implisit.js`, `konversi-eksplisit.js`
Function	`function.js`, `return-value.js`, `function-experession.js`, `arrow-function.js`
Function sebagai nilai	`first-class-citizen.js`, `struktur-data/array-object-function.js`
Array	`struktur-data/array.js`, `manipulasi-array.js`, `array-method.js`, `array-destructuring.js`
Object	`struktur-data/object.js`
Map	`struktur-data/map.js`, `real-case-map.js`
DOM dan event	`dom/index.html`, `dom/event.html`
Modul	`modularisasi/`, `ESModule/`
Latihan serupa diskon	`challange.js`, `arrow-function.js` (hitungNilai)

Lampiran C: Istilah

Istilah	Arti singkat
Render	Membuat tampilan (HTML) dari data
State	Data yang berubah selama halaman terbuka (kategori aktif, kupon aktif)
Callback	Function yang dikirim ke function lain untuk dijalankan nanti (`forEach`, `addEventListener`, `setTimeout`, `sort`)
Closure	Function yang masih "mengingat" variabel dari tempat ia dibuat (`buatKupon`)
Module	File JavaScript yang memakai `import` dan `export`
Destructuring	Mengambil isi object atau array ke variabel terpisah
Pseudo-element	Elemen buatan CSS (`::before`, `::after`)
Media query	Aturan CSS yang hanya berlaku pada kondisi layar tertentu

Rubrik dan Lembar Penilaian: Proyek "Lapak Buah 11-A"

Dokumen untuk guru. Tujuan penilaian bukan sekadar "websitenya jalan", tetapi **apakah siswa memahami materi yang dipakai**. Karena itu seperempat nilai berasal dari wawancara kode dan tugas ubah-langsung.

---

1. Komponen dan bobot

Komponen	Aspek	Bobot
A. Proses dan kemandirian (15)	A1 Checkpoint harian	5
	A2 Exit ticket	5
	A3 Kemandirian, kerja sama, kejujuran akademik	5
B. Produk (50)	B1 Fungsionalitas fitur wajib	10
	B2 Variabel, operator, konversi tipe	5
	B3 Function dan function sebagai nilai	8
	B4 Array dan object	5
	B5 Map	4
	B6 DOM dan event	6
	B7 ES Module	4
	B8 CSS native dan responsif	8
C. Pemahaman (25)	C1 Menjelaskan kode sendiri	10
	C2 Pertanyaan konsep	5
	C3 Ubah-langsung	10
D. Kualitas dan refleksi (10)	D1 Kerapian kode	5
	D2 Refleksi tertulis	5
	Total	**100**

2. Skala dan cara menghitung

Level	Nama	Persentase bobot
4	Sangat baik	100%
3	Baik	75%
2	Cukup	50%
1	Perlu bimbingan	25%
0	Belum ada	0%

Skor aspek = bobot x level / 4. Skor akhir adalah jumlah seluruh skor aspek (maksimum 100).

Contoh: seorang siswa mendapat level 3 di semua aspek. Skor akhirnya 100 x 3 / 4 = **75**, tepat di KKM. Artinya, level 3 ("baik") berarti memenuhi kriteria, dan nilai di atas KKM membutuhkan sebagian aspek di level 4.

Konversi nilai. Memakai skala yang sama dengan `tentukanPredikat` di `ESModule/config.mjs`.

Skor	Predikat	Keterangan
90 sampai 100	A	Sangat baik
80 sampai 89	B	Baik
70 sampai 79	C	Cukup
60 sampai 69	D	Kurang
di bawah 60	E	Perlu perbaikan

KKM 75 (sama dengan batas lulus pada latihan `hitungNilai` di `arrow-function.js`). Ubah sesuai kebijakan sekolah.

---

3. Rubrik rinci

A. Proses dan kemandirian

Aspek	4 Sangat baik	3 Baik	2 Cukup	1 Perlu bimbingan
A1 Checkpoint harian (5 checkpoint, tabel di bagian 5)	5 dari 5 tercapai tepat waktu	4 dari 5	3 dari 5	2 atau kurang
A2 Exit ticket (5 kali)	Rata-rata benar 85% atau lebih	70 sampai 84%	50 sampai 69%	di bawah 50% atau tidak dikumpulkan
A3 Kemandirian dan kejujuran	Menulis kode sendiri, bertanya dengan urutan "baca error, teman, guru", membantu teman tanpa menyalin	Menulis sendiri, sesekali langsung bertanya guru	Sering meminta jawaban, kode sebagian mirip teman tanpa bisa menjelaskan	Menyalin kode yang tidak bisa dijelaskan

B. Produk

Bukti utama: hasil `node uji-logika.mjs` (tes otomatis), pemeriksaan halaman di browser, dan membaca kode.

Aspek	4 Sangat baik	3 Baik	2 Cukup	1 Perlu bimbingan
B1 Fungsionalitas fitur wajib (bobot 10)	Tes **WAJIB** lulus semua (25 dari 25), seluruh fitur wajib berjalan, Console tanpa error	Tes WAJIB 21 sampai 24 lulus, fitur wajib berjalan dengan 1 sampai 2 bug kecil	Tes WAJIB 13 sampai 20 lulus. Katalog dan nota berjalan, diskon atau pembayaran belum benar	Tes WAJIB kurang dari 13. Hanya katalog yang tampil
B2 Variabel, operator, konversi (5)	`const` dan `let` tepat. `Number()` untuk input. `%`, `++`, `-=`, ternary dipakai benar. Angka aturan memakai konstanta	Benar, tetapi masih ada 1 sampai 2 angka aturan ditulis langsung (magic number)	`Number()` dipakai, namun operator banyak yang keliru atau konstanta tidak dipakai	Hasil sering `NaN` atau string karena konversi tidak dilakukan
B3 Function dan function sebagai nilai (8)	Function declaration, expression, dan arrow dipakai sesuai tujuan. Parameter default. `return` konsisten. Kupon (Level 2) berjalan dengan closure dan Map berisi function	Ketiga bentuk dipakai dan `return` konsisten. Kupon belum dikerjakan	Function ada, sebagian hasilnya lewat `console.log` atau langsung ke DOM bukan `return`	Sebagian besar logika ditulis di luar function
B4 Array dan object (5)	`data.js` valid (tes DATA 4 dari 4). Destructuring di 2 tempat atau lebih. `Array.from` sebelum `sort` atau `reverse`. Method array tepat	Data valid. Destructuring di 1 tempat	Data valid, tetapi mengubah array asli tanpa sadar atau tidak memakai destructuring	Data tidak valid (tes DATA gagal)
B5 Map (4)	`Map` dengan key object. `has`, `get`, `set`, `delete` tepat. Tes KERANJANG 9 dari 9	Tes KERANJANG 7 sampai 8	`Map` dipakai tetapi key berupa string atau logika banyak yang menyimpang	Tidak memakai `Map`
B6 DOM dan event (6)	Katalog dan nota dirender dari data. Event `click`, `input`, `change` benar. Pola "render lalu pasang event" konsisten. `textContent`, `className` dipakai sesuai tujuan. Tidak ada error `null`	Berfungsi, tetapi ada duplikasi kode atau elemen yang tidak perlu dibuat ulang	Sebagian elemen ditulis tetap di HTML dan tidak mengikuti data	Halaman tidak interaktif
B7 ES Module (4)	4 file module atau lebih. Named export, default export, dan `import * as` dipakai. Tidak ada variabel global. Berjalan lewat server	4 file module atau lebih, dua jenis export/import	2 sampai 3 file module	Semua kode dalam satu file
B8 CSS native dan responsif (8)	Variabel CSS, grid dan flex, 3 media query, state `.habis`, mode gelap. Tampil benar di 375 px dan 1366 px	Tampil benar di kedua lebar, kekurangan satu teknik (misalnya mode gelap)	Tampilan jadi tetapi tidak responsif (scroll mendatar di HP)	Banyak blok TODO CSS masih kosong

Catatan Jalur Kejar: siswa yang menyalin `css/style.css` dari `contoh-jadi/` dengan izin guru mendapat **maksimal level 2** pada B8. Aspek lain tetap dinilai dari kode yang ditulisnya sendiri.

C. Pemahaman (wawancara kode)

Dilakukan di Pertemuan 5, sekitar 2 menit per siswa. Panduan pelaksanaan dan kartu pertanyaan ada di `05-bahan-guru.md`, bagian C.

Aspek	4 Sangat baik	3 Baik	2 Cukup	1 Perlu bimbingan
C1 Menjelaskan kode sendiri (10)	Menjelaskan 3 fungsi pilihan guru (masukan, proses, keluaran, tempat dipakai) dengan benar tanpa membaca kode baris per baris	Menjelaskan 2 fungsi dengan benar	Menjelaskan dengan membaca baris per baris, benar sebagian	Tidak dapat menjelaskan
C2 Pertanyaan konsep (5)	Menjawab benar dan memberi contoh sendiri	Menjawab benar	Menjawab sebagian	Menjawab salah atau tidak menjawab
C3 Ubah-langsung (10)	Perubahan benar dalam 2 menit, langsung tahu file dan baris yang diubah	Benar dengan 1 petunjuk	Menemukan lokasi yang tepat tetapi perubahannya belum benar	Tidak dapat menemukan lokasinya

D. Kualitas kode dan refleksi

Aspek	4 Sangat baik	3 Baik	2 Cukup	1 Perlu bimbingan
D1 Kerapian kode (5)	Nama camelCase yang bermakna (sesuai kesepakatan kelas), indentasi konsisten, komentar menjelaskan alasan (bukan mengulang kode), tidak ada kode mati atau `console.log` sisa	Rapi, ada 1 sampai 2 sisa `console.log` atau komentar kurang bermakna	Penamaan tidak konsisten atau indentasi berantakan	Sulit dibaca
D2 Refleksi tertulis (5)	Menyebut bagian yang dipahami disertai bukti (satu potongan kode), bagian yang membingungkan, dan rencana belajar yang konkret	Memuat dua dari tiga unsur	Umum dan tanpa bukti ("saya paham semua")	Tidak mengumpulkan

---

4. Contoh pemeriksaan cepat B1 (5 menit per siswa)

1. Buka folder siswa lewat Live Server. Console tanpa error merah?
2. Jalankan `node uji-logika.mjs`. Catat angka WAJIB, LEVEL 2, BONUS.
3. Jalankan skenario cepat:
   - Apel 3 kg, Anggur 1 kg: subtotal Rp156.000, diskon 10% Rp15.600, total Rp140.400
   - Apel 1 kg, Pisang 1 kg, Mangga 1 kg: subtotal Rp75.000, diskon Rp3.750, pembulatan Rp50, total Rp71.200
   - Bayar Rp100.000 untuk total Rp140.400: "Uang kurang Rp40.400"
4. Resize ke lebar 375 px: tidak ada scroll mendatar.

(Angka di atas berlaku jika siswa memakai data yang direkomendasikan di langkah 1.6 panduan siswa.)

---

5. Lembar penilaian kelas

5.1 Tabel checkpoint harian (A1)

Centang jika tercapai pada hari itu.

Nama	P1: tes DATA 4/4 dan halaman tertata	P2: kartu tampil dari data, "Habis" otomatis	P3: tes KERANJANG 9/9, tombol Tambah berfungsi	P4: total dan kembalian benar	P5: fitur wajib lengkap dan uji silang selesai	Jumlah

5.2 Tabel skor akhir

Isi level (0 sampai 4) pada setiap kolom. Baris "Bobot" dibiarkan tetap.

Nama	A1	A2	A3	B1	B2	B3	B4	B5	B6	B7	B8	C1	C2	C3	D1	D2	Skor	Predikat
Bobot	5	5	5	10	5	8	5	4	6	4	8	10	5	10	5	5	100	

Rumus spreadsheet. Misal baris Bobot di baris 2 (kolom B sampai Q) dan level siswa di baris 3 (kolom B sampai Q):

```
=SUMPRODUCT(B3:Q3; B$2:Q$2) / 4
```

Gunakan titik koma atau koma sesuai pengaturan spreadsheet.

Versi JavaScript (opsional, gaya yang sama dengan `ESModule/config.mjs`).

```
const bobot = [5, 5, 5, 10, 5, 8, 5, 4, 6, 4, 8, 10, 5, 10, 5, 5];

const hitungSkor = (level) => {
  const total = level.reduce((jumlah, nilai, index) => jumlah + (nilai * bobot[index]) / 4, 0);
  return total;
};

const tentukanPredikat = (nilai) => {
  if (nilai >= 90) return "A";
  if (nilai >= 80) return "B";
  if (nilai >= 70) return "C";
  if (nilai >= 60) return "D";
  return "E";
};

console.log(hitungSkor([3, 3, 3, 4, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3])); // 77.5
```

5.3 Lembar wawancara kode (C1 sampai C3)

Nama	Fungsi yang diminta (3)	C1	Pertanyaan konsep (nomor di bank soal)	C2	Tugas ubah-langsung (nomor)	C3	Catatan

5.4 Lembar observasi harian (opsional)

Tanggal	Pertemuan	Kondisi kelas	Siswa yang perlu perhatian	Tindak lanjut

6. Tindak lanjut hasil

Hasil	Tindak lanjut
Skor 75 ke atas	Pengayaan: kupon sekali pakai, simpan nota ke `localStorage` (materi baru), atau membantu teman (tutor sebaya).
Skor 60 sampai 74	Perbaikan: memperbaiki aspek yang kurang (daftar dari rubrik), lalu wawancara ulang 3 menit.
Skor di bawah 60	Remedial terbimbing: mengerjakan ulang langkah 3.1 dan 4.1 sampai tes WAJIB lulus, lalu wawancara ulang.

7. Kejujuran akademik

- Panduan sengaja memakai contoh dari kasus lain agar siswa menerjemahkan sendiri. Wawancara dan ubah-langsung adalah penentu skor aspek C.
- Kode yang mirip belum tentu menyalin, karena langkahnya memang sama. Yang dinilai: apakah siswa bisa menjelaskan dan mengubahnya.
- Siswa yang memakai bantuan AI atau teman untuk menulis fungsi, tetapi tidak dapat menjelaskannya, mendapat C1 dan C3 pada level 1, dan A3 diturunkan. Ini bukan hukuman, melainkan cerminan pemahaman yang sebenarnya.

Lembar Kerja Siswa: Proyek "Lapak Buah 11-A"

Nama: _____ No. absen: _____ Kelas: 11-A

Isi lembar ini sambil mengerjakan `02-panduan-siswa.md`. Tulis jawaban dengan kata-katamu sendiri. **Bagian "Tebak" diisi SEBELUM menjalankan kode.**

Pertemuan 1: Senin 19 Oktober 2026

Tebakan pembuka

Dari demo guru, fitur mana yang menurutmu memakai materi apa?

Fitur yang kamu lihat	Materi yang dipakai (tebakanmu)
1.	
2.	
3.	

Langkah 1.1 sampai 1.2

Tebak: jika `index.html` dibuka dengan klik dua kali, apa yang terjadi?

Jawab: kenapa halaman dengan `<script type="module">` harus dibuka lewat server?

Peta ID (minimal 8 baris):

id	Jenis elemen	JavaScript akan melakukan apa padanya?
`#sapaan`		

id	Jenis elemen	JavaScript akan melakukan apa padanya?

Jawab: apa yang terjadi di `main.js` jika id `#tombol-bayar` di HTML diganti `#tombol-bayar2`?

Langkah 1.3 sampai 1.5

Tebak (1.3): `justify-content` mengatur arah mendatar atau menurun? `align-items`?

Jawab (1.4): apa beda `minmax(0, 1fr)` dan `1fr`?

Tebak (1.5): jumlah kolom `repeat(auto-fill, minmax(150px, 1fr))` (jarak 12 px) di layar 600 px: _____ kolom. Di layar 300 px: _____ kolom.

Jawab (1.5): kenapa `.kartu-foto` harus `position: relative`? Dari mana warna latar fotonya diambil?

Langkah 1.6

Tebak: hasil `"18.000" * 2` adalah _____ . (Cek di Console. Sesuai tebakanmu?)

Jawab: kenapa `id` setiap produk harus unik?

Checkpoint Pertemuan 1 (diisi guru)

- ☐ Halaman terbuka lewat `http://` dan Console tanpa error
- ☐ Header, papan, dan dua kolom tampil
- ☐ Kartu contoh rapi dan state "habis" bisa dicoba
- ☐ `node uji-logika.mjs`: bagian DATA 4 dari 4 ✅

Paraf guru: _____

Exit ticket Pertemuan 1

1. Kenapa proyek ini harus dibuka lewat Live Server, bukan klik dua kali?

---

2. Tulis satu object produk dengan key `name`, `price`, `category`, `stok`.

---

3. Sebutkan dua keuntungan variabel CSS (`--aksi`) dibanding menulis warna langsung.

Pertemuan 2: Selasa 20 Oktober 2026 (Jam ke-3)

Tebak dan jawab

Tebak (2.1): `(32000).toLocaleString("id-ID")` menghasilkan _____ dengan tipe data _____ .

Jawab (2.1): apa beda function expression dan function declaration saat dipanggil sebelum barisnya?

Tebak (2.2): tipe data hasil `buatBarisSiswa(...)` (contoh di panduan): _____

Jawab (2.2): kenapa `buatKartu` mengembalikan string dan tidak langsung mengisi halaman?

Tebak (2.3): jika `let html;` (tanpa `= ""`), isi `html` di akhir diawali teks _____

Jawab (2.3): apa yang terjadi bila event dipasang sebelum `katalog.innerHTML = html`? Tulis pesan errornya.

Checkpoint Pertemuan 2 (diisi guru)

- [ ] `formatRupiah` lulus tes
- [ ] Kartu tampil dari `data.js` (bukan dari HTML)
- [ ] Produk `stok: 0` otomatis "Habis" setelah halaman dimuat ulang

Paraf guru: _____

Exit ticket Pertemuan 2

1. `buatKartu(item)` mengembalikan tipe data apa? _____
2. Kenapa tombol di kartu tidak bisa diberi `addEventListener` sebelum `innerHTML` diisi?

---

3. `formatRupiah()` tanpa argumen menghasilkan _____ . Fitur apa yang membuatnya begitu? _____

Pertemuan 3: Selasa 20 Oktober 2026 (Jam ke-6 dan 7)

Tebak dan jawab

Tebak (3.1):

```
const a = { id: 1 };
const b = { id: 1 };
const peta = new Map();
peta.set(a, "x");
console.log(peta.has(a), peta.has(b), peta.size);
```

Hasil: _____ , _____ , _____

Jawab: kenapa `peta.has(b)` bernilai `false`?

Jawab: apa urutan parameter callback `Map.forEach`? Apa bedanya dengan `Array.forEach`?

Jawab: apa yang terjadi bila `tambahItem` tidak mengecek stok?

Jawab (3.2): kenapa parameter `keranjang.forEach` ditulis `(jumlah, item)`?

Uji manual nota (Langkah 3.3)

No	Skenario	Berhasil?
1	Tambah Apel 3 kali, jadi 3 kg, badge 3	
2	Tombol `−` pada Apel, jadi 2 kg	
3	Tambah Mangga (stok 4) 5 kali, berhenti di 4 kg dan ada pesan	
4	`−` pada item 1 kg, item hilang	
5	"Hapus" pada item 3 kg, hilang sekaligus	
6	Produk stok 0, tombol nonaktif	

Checkpoint Pertemuan 3 (diisi guru)

- ☐ `node uji-logika.mjs`: bagian KERANJANG 9 dari 9 ✅
- ☐ Tombol "Tambah ke nota", `−`, `+`, "Hapus" berfungsi
- ☐ Jumlah tidak bisa melebihi stok

Paraf guru: _____

Exit ticket Pertemuan 3

1. Kenapa `peta.has(objekBaruIsiSama)` bernilai `false`?

---

2. Tulis urutan parameter callback `Map.forEach`: `(_____, _____)`
3. Apa risikonya jika `tambahItem` tidak memeriksa stok?

Pertemuan 4: Rabu 21 Oktober 2026

Tebak dan hitung dengan tangan (SEBELUM menulis kode)

Tabel diskon (4.1):

Subtotal	Persen diskon	Diskon (Rp)
60.000		
74.999		
75.000		
150.000		

Tabel ringkasan (4.2):

Subtotal	Kupon	Diskon	Potongan kupon	Sebelum pembulatan	Pembulatan	Total
100.000	tidak ada					
77.000	tidak ada					
80.000	10% (Level 2)					

Tebak (4.2): `73150 % 100` = _____ . `((a, f = () => 0) => f(a))(5)` = _____

Tebak (4.3): jika `inputBayar.value` berisi `100000`:

Ekspresi	Hasil tebakanmu	Hasil sebenarnya
`inputBayar.value + 1000`		
`Number(inputBayar.value) + 1000`		
`inputBayar.value – 1000`		
`Number("")`		

Jawab (4.1): apa bedanya `3 * 5 / 100` dan `3 * (5 / 100)` di Console? Kenapa urutan pertama lebih aman untuk uang?

Jawab (4.3): apa beda `addEventListener("input", renderKembalian)` dengan `addEventListener("input", renderKembalian())`?

Jawab (4.4): kenapa stok dikurangi setelah pembayaran, bukan saat item masuk nota?

Jawab (4.5, Level 2): kenapa `daftarKupon.get("HEMAT10")(200000)` bisa langsung dipanggil seperti function?

Uji pembayaran (Langkah 4.4)

Skenario	Berhasil?
Bayar saat nota kosong: pesan nota kosong	
Bayar kurang: pesan uang kurang, nota tetap	
Bayar cukup: kembalian benar, nota kosong, stok kartu berkurang	
Beli semua stok Mangga lalu bayar: kartu Mangga menjadi "Habis"	

Checkpoint Pertemuan 4 (diisi guru)

- ☐ `node uji-logika.mjs`: seluruh tes WAJIB ✅ (25 dari 25)
- ☐ Ringkasan nota (subtotal, diskon, pembulatan, total) benar
- ☐ Kembalian dan "uang kurang" benar

Paraf guru: _____

Exit ticket Pertemuan 4

1. `inputBayar.value` bernilai `"100000"`. Hasil `inputBayar.value + 1000` adalah _____ dan `Number(inputBayar.value) + 1000` adalah _____
2. Tulis ternary: diskon 10% jika subtotal >= 150.000, 5% jika >= 75.000, selain itu 0.

3. `73150 % 100` = _____ . Dipakai untuk apa di nota?

Pertemuan 5: Kamis 22 Oktober 2026

Uji silang (Langkah 5.3)

Teman yang kamu uji: _____

Pilih Berhasil (✔) atau Gagal (✘). Jika gagal, tulis apa yang terjadi di kolom catatan. **Jangan memperbaiki karya temanmu.**

No	Skenario	Hasil yang benar	✔/✘	Catatan
1	Buka lewat server, lihat Console	Tidak ada error merah		
2	Lihat katalog	Minimal 6 kartu. Produk stok 0 berlabel "Habis", tombolnya nonaktif		

No	Skenario	Hasil yang benar	✔/✘	Catatan
3	Tambah Apel 2 kali	Nota: Apel 2 kg. Badge Nota menjadi 2		
4	Klik `–` dua kali pada Apel	2 menjadi 1, lalu Apel hilang dari nota		
5	Tambah Mangga (stok 4) 5 kali	Berhenti di 4 kg, ada pesan stok		
6	Apel 3 kg, Anggur 1 kg	Subtotal Rp156.000, diskon 10%, total Rp140.400		
7	Apel 1 kg, Pisang 1 kg, Mangga 1 kg	Subtotal Rp75.000, diskon Rp3.750, pembulatan Rp50, total Rp71.200		
8	Kupon `hemat10` (huruf kecil) lalu kupon `ngawur` (Level 2)	Kupon pertama diterima, kedua menampilkan pesan merah		
9	Bayar kurang, lalu bayar lebih	Pesan "uang kurang", lalu kembalian benar		
10	Setelah bayar	Nota kosong, stok di kartu berkurang		
11	Klik "Mode gelap" dua kali	Gelap, lalu kembali terang. Teks tombol ikut berubah		
12	Lebar layar 375 px (DevTools, `Ctrl + Shift + M`)	Tanpa scroll mendatar. Nota di bawah katalog		

(Angka skenario 6 dan 7 berlaku jika data produk sama dengan tabel di langkah 1.6.)

Bug dari uji silang yang harus kuperbaiki

No skenario	Apa yang terjadi	Sudah diperbaiki?

Persiapan wawancara kode

Pilih tiga fungsi yang kamu tulis sendiri. Isi dengan kata-katamu:

Fungsi (nama dan file)	Menerima	Melakukan	Mengembalikan	Dipakai di
1.				
2.				
3.				

Centang fitur yang sudah kamu selesaikan

Wajib

- ☐ Katalog dari `data.js` (minimal 6 produk)

- [ ] Tambah ke nota dengan `Map`, ubah jumlah, hapus
- [ ] Subtotal, diskon bertingkat, pembulatan, total
- [ ] Bayar dan kembalian (`Number()`)
- [ ] Kode terpecah ke module ES
- [ ] CSS responsif dan mode gelap
- [ ] Sapaan dan papan info

Level 2

- [ ] Kupon (`Map` berisi function, `buatKupon`)

Bonus

- [ ] Filter dan urutan katalog
- [ ] Riwayat pesanan
- [ ] Ganti warna apel

Jelaskan 3 aturan CSS di bagian nota (yang disediakan guru)

Aturan	Menurutku fungsinya
`filter: drop-shadow(...)` pada `.nota`	
`conic-gradient(...)` pada `.nota::after`	
`grid-template-areas` pada `.baris-nota`	

Refleksi (3 sampai 5 kalimat)

1. Bagian proyek yang paling kupahami, dan buktinya (tulis satu potongan kode):

2. Bagian yang masih membingungkan:

3. Rencanaku mempelajarinya:

Exit ticket Pertemuan 5

Jika aku ingin menambah fitur "kupon sekali pakai", file mana yang kuubah dan kenapa?

Checkpoint Pertemuan 5 (diisi guru)

- [ ] Fitur wajib lengkap
- [ ] Uji silang selesai dan bug ditangani
- [ ] Wawancara kode selesai

Paraf guru: _____

Bahan Guru: Mini-lesson, Kunci Jawaban, Bank Soal, Troubleshooting

Dokumen untuk guru. Siswa tidak perlu memegang file ini.

Isi:

- **A.** Mini-lesson M1 sampai M8 (naskah ringkas, contoh kode, kunci)
- **B.** Pemanasan dan exit ticket per pertemuan (soal dan kunci)
- **C.** Wawancara kode: cara pelaksanaan, bank pertanyaan konsep, tugas ubah-langsung
- **D.** Kunci jawaban pertanyaan di panduan siswa
- **E.** Troubleshooting
- **F.** Peta kode `contoh-jadi/`

A. Mini-lesson (materi jembatan)

Setiap mini-lesson dibuat singkat. Pola: **tebak, lihat hasilnya, jelaskan sebabnya**.

M1. Server lokal dan `<script type="module">` (Pertemuan 1, 5 menit)

Tujuan: siswa tahu kenapa halaman harus dibuka lewat `http://`, bukan `file://`.

Peragaan:

1. Klik dua kali `index.html` starter. Alamat berawalan `file:///`. Buka Console (`F12`): muncul pesan merah `Access to script at 'file:///...' from origin 'null' has been blocked by CORS policy`.
2. Buka lewat Live Server. Alamat `http://127.0.0.1:5500/...`. Pesan error hilang.

Penjelasan:

- Browser menganggap halaman dari `file://` berasal dari "origin null" dan memblokir pemuatan module demi keamanan. Server lokal membuat halaman berasal dari origin `http://localhost`.
- Di folder `ESModule/` module jalan lewat Node karena tidak ada sandbox browser, jadi tidak butuh server.

	`<script src="x.js">`	`<script type="module" src="x.js">`
`import` dan `export`	tidak bisa	bisa
Variabel di file	menjadi global	hanya milik file itu
Waktu berjalan	langsung saat dibaca	setelah HTML selesai dibaca (otomatis `defer`)
Strict mode	tidak otomatis	otomatis
Dibuka dengan klik dua kali	bisa	tidak bisa

Kunci tebak ("dibuka klik dua kali, jalan?"): tidak. Tampilan HTML dan CSS muncul, tetapi JavaScript tidak jalan.

M2. Variabel CSS, box model, flex dan grid (Pertemuan 1, 5 menit)

Contoh:

```
:root { --utama: #1b6b3c; }          /* sekali definisi */
.tombol { background: var(--utama); } /* dipakai di banyak tempat */
body.gelap { --utama: #5fd18a; }      /* menimpa nilai: semua yang memakainya ikut berubah */
```

Box model: `content`, `padding`, `border`, `margin`. Tanpa `box-sizing: border-box`, lebar elemen = `width + padding + border`.

Tebak: elemen `width: 200px; padding: 20px; border: 2px solid`. Lebar totalnya?

- Tanpa `border-box`: 200 + 40 + 4 = **244 px**.
- Dengan `border-box`: **200 px** (padding dan border masuk ke dalam).

Flex dan grid:

	Flex	Grid
Arah	satu sumbu (baris atau kolom)	dua sumbu (baris dan kolom)
Cocok untuk	susunan di dalam satu komponen (header, baris nota)	kerangka halaman, daftar kartu
Contoh di proyek	`.header-isi`, `.stepper`	`.tata-letak`, `.katalog`

M3. `innerHTML` dan template literal (Pertemuan 2, 4 menit)

Contoh:

```
const siswa = { nama: "Dimas", nilai: 80 };
console.log("Halo " + siswa.nama + ", nilai " + siswa.nilai);   // cara lama
console.log(`Halo ${siswa.nama}, nilai ${siswa.nilai}`);       // template literal
console.log(`Status: ${siswa.nilai >= 75 ? "lulus" : "remedial"}`); // ekspresi apa pun boleh
```

Peragaan `innerHTML` dan `textContent`:

```
div.innerHTML = "<b>tebal</b>";     // tampil: tebal  (HTML diproses)
div.textContent = "<b>tebal</b>";   // tampil: <b>tebal</b>  (apa adanya sebagai teks)
```

Catatan keamanan untuk guru: `innerHTML` hanya aman untuk data yang kita kontrol (isi `data.js`). Teks yang diketik pengguna (misalnya kode kupon) selalu ditampilkan dengan `textContent`. Sampaikan satu kalimat saja: "teks dari pengguna jangan dimasukkan ke `innerHTML`, karena bisa berisi kode berbahaya."

M4. Pola "render, lalu pasang event" (Pertemuan 2, 3 menit)

Peragaan error `null`:

```
const tombol = document.querySelector("#btn-hapus");   // null, elemennya belum ada
tombol.addEventListener("click", () => {});           // TypeError: Cannot read properties of
        null
daftar.innerHTML = '<button id="btn-hapus">Hapus</button>';
```

Penjelasan: urutan yang benar: isi `innerHTML` dulu, baru `querySelector`, baru `addEventListener`. Karena setiap render membangun ulang elemen, event dari render sebelumnya hilang, jadi event dipasang ulang setiap kali render.

M5. Map dengan key object (Pertemuan 3, 8 menit)

Contoh (mirip `real-case-map.js`):

```
const sesi = new Map();
const userA = { id: 1, nama: "Rizky" };

sesi.set(userA, "Login");
console.log(sesi.has(userA));                       // true
console.log(sesi.has({ id: 1, nama: "Rizky" }));    // false: object baru, alamat memori
        berbeda
console.log(sesi.size);                             // 1

sesi.forEach((status, user) => console.log(user.nama, status));   // (value, key)
console.log(Array.from(sesi));                      // [[ {id:1,...}, "Login" ]]
```

Poin penjelasan:

1. Object dibandingkan berdasarkan **alamat**, bukan isi. Dua object dengan isi sama tetap dua object.
2. `Map.forEach` memberi `(value, key)`, kebalikan urutan "key dulu" yang biasa kita bayangkan, dan berbeda dari `Array.forEach (item, index)`.
3. `Array.from(map)` menghasilkan array berisi pasangan `[key, value]`. Dipakai di proyek untuk menyimpan isi nota ke pesanan.

Kunci tebak: `true`, `false`, `1`.

M6. `input.value` selalu string (Pertemuan 4, 7 menit)

Tebak dulu:

```
const input = "100000";       // seperti inputBayar.value
console.log(input + 1);           // ?
console.log(Number(input) + 1);   // ?
console.log(input - 1);           // ?
console.log(Number(""));          // ?
console.log(Number("12abc"));     // ?
```

Kunci: `"1000001"`, `100001`, `99999`, `0`, `NaN`.

Penjelasan:

- `+` bila salah satu operannya string: menggabungkan (`konversi-implisit.js`: `"40" + 40` menjadi `"4040"`).
- `-` selalu aritmetika, sehingga string diubah ke number secara implisit.
- `Number("")` bernilai 0: kolom bayar kosong dianggap Rp0, sehingga pemeriksaan "uang kurang" tetap benar.
- Ulas ternary bersusun dengan `challange.js`: `gaji >= 5000000 ? ... : gaji >= 3000000 ? ... : 0`.

M7. Function yang mengembalikan function (Pertemuan 4, 5 menit; Level 2)

Hubungkan ke `multiplier` di `first-class-citizen.js`.

```
const multiplier = (x) => (num) => x * num;     // bentuk arrow dari contoh di kelas
const double = multiplier(2);
double(10);                                      // 20

const buatPajak = (persen) => (harga) => (harga * persen) / 100;
const ppn11 = buatPajak(11);                     // ppn11 adalah function
ppn11(100000);                                   // 11000
```

Poin:

- `buatPajak(11)` menghasilkan function baru yang "mengingat" angka 11. Istilahnya closure.
- Sama dengan `double` yang mengingat angka 2.
- Dengan Map: `new Map([["PPN11", buatPajak(11)]])`, lalu `daftar.get("PPN11")(100000)`. Function bisa disimpan di Map seperti data lain.

Kunci tebak ("di mana angka 11 disimpan?"): di function yang dikembalikan, yang membawa variabel `persen` dari tempat ia dibuat.

M8. `sort` dan `filter` menerima function (Pertemuan 5, 5 menit; Bonus)

```
const angka = [10, 9, 100, 1];
angka.sort();                       // [1, 10, 100, 9]  default: diurutkan sebagai STRING
angka.sort((a, b) => a - b);        // [1, 9, 10, 100]  dengan function pembanding

const produkMurah = produk.filter((item) => item.price < 25000);   // array baru
```

Poin:

- `sort()` tanpa argumen mengurutkan sebagai teks (`array-method.js`). Untuk angka dan object butuh function pembanding.
- Function pembanding menerima `(a, b)`. Hasil negatif: `a` di depan. Positif: `b` di depan.
- `sort` mengubah array aslinya, karena itu salin dulu dengan `Array.from`.
- `filter` tidak mengubah array asli, mengembalikan array baru.
- Untuk teks: `(a, b) => (a.name > b.name ? 1 : -1)`.

Kunci tebak: `[1, 10, 100, 9]`, lalu `[1, 9, 10, 100]`.

B. Pemanasan dan exit ticket

Waktu exit ticket 3 sampai 5 menit, dijawab di lembar kerja. Periksa sambil berkeliling atau kumpulkan.

Pemanasan Pertemuan 2

Soal	Kunci
`const { nama, kelas } = { nama: "Rizky", kelas: 11, hobi: "bola" }; console.log(nama, kelas);`	`Rizky 11`
`["a", "b"].forEach((x, i) => console.log(i, x));`	`0 a` lalu `1 b`
`const n = 3; console.log(`Total: ${n * 2} item`);`	`Total: 6 item`

Exit ticket Pertemuan 1

Soal	Kunci
1. Kenapa proyek ini harus dibuka lewat Live Server, bukan klik dua kali?	Karena memakai `<script type="module">`. Browser memblokir module dari `file://`.
2. Tulis satu object produk dengan key name, price, category, stok.	Contoh: `{ name: "Apel", price: 32000, category: "Impor", stok: 12 }`. Harga number tanpa titik.
3. Sebutkan dua keuntungan variabel CSS (`--aksi`) dibanding menulis warna langsung.	Ubah di satu tempat berlaku di semua. Mode gelap cukup menimpa nilai. Konsisten.

Exit ticket Pertemuan 2

Soal	Kunci
1. `buatKartu(item)` mengembalikan tipe data apa?	String (HTML dalam template literal).
2. Kenapa tombol di kartu tidak bisa diberi `addEventListener` sebelum `innerHTML` diisi?	Elemennya belum ada. `querySelector` mengembalikan `null`, sehingga muncul `TypeError`.
3. `formatRupiah()` tanpa argumen menghasilkan apa, dan fitur apa yang menyebabkannya?	`"Rp0"`. Parameter default (`angka = 0`).

Exit ticket Pertemuan 3

Soal	Kunci
1. Kenapa `peta.has(objekBaruIsiSama)` bernilai `false`?	Object dibandingkan berdasarkan alamat (referensi), bukan isi.
2. Tulis urutan parameter callback `Map.forEach`. Apa bedanya dengan `Array.forEach`?	`(value, key)`. `Array.forEach` memakai `(item, index)`.
3. Apa risikonya bila `tambahItem` tidak memeriksa stok?	Pembeli bisa membeli lebih banyak dari stok yang ada.

Pemanasan Pertemuan 4

Soal	Kunci
`"100000" + 1`	`"1000001"`
`"100000" - 1`	`99999`
`Number("")`	`0`

Exit ticket Pertemuan 4

Soal	Kunci
1. `inputBayar.value` bernilai `"100000"`. Hasil `inputBayar.value + 1000` dan `Number(inputBayar.value) + 1000`?	`"1000001000"` dan `101000`.
2. Tulis ternary: diskon 10% jika subtotal >= 150.000, 5% jika >= 75.000, selain itu 0.	`subtotal >= 150000 ? 10 : subtotal >= 75000 ? 5 : 0`
3. Berapa `73150 % 100`, dan untuk apa dipakai di nota?	`50`. Pembulatan total ke bawah kelipatan Rp100.

Exit ticket Pertemuan 5 (refleksi)

Soal	Catatan penilaian
1. Bagian proyek yang paling kamu pahami. Tulis satu potongan kode sebagai buktinya.	Bukti berupa kode konkret.
2. Bagian yang masih membingungkan dan rencanamu.	Rencana konkret ("baca ulang `map.js` lalu ulangi langkah 3.1").
3. Jika ingin menambah fitur "kupon sekali pakai", file mana yang diubah dan kenapa?	`helper.js` (aturan kupon) dan `main.js` (tampilan dan pemanggilan), karena logika dan tampilan dipisah.

C. Wawancara kode (Pertemuan 5)

Cara pelaksanaan (sekitar 2 menit per siswa)

1. Siswa membuka karyanya. Guru menyebut **3 fungsi** yang harus dijelaskan (pilih dari yang tersulit yang ia tulis: misalnya `tambahItem`, `hitungRingkasan`, `buatKartu`).
2. Guru mengajukan **1 pertanyaan konsep** dari bank di bawah.
3. Guru memberi **1 tugas ubah-langsung** (maksimal 2 menit).
4. Guru mengisi tabel 5.3 di `03-rubrik-penilaian.md`.

Tips: jangan menilai kefasihan berbahasa. Nilai ketepatan. Beri jeda berpikir 10 detik sebelum memberi petunjuk. Siswa yang menjelaskan dengan membaca baris per baris (`"ini const, ini kurung..."`) belum paham alurnya, mintalah: "ceritakan seolah-olah ke teman yang belum belajar."

Pola penjelasan yang diharapkan: "*Fungsi ini menerima ..., melakukan ..., mengembalikan ... . Dipakai di ... .*"

Bank pertanyaan konsep dan kunci

No	Topik	Pertanyaan	Kunci ringkas
Q1	Variabel	Apa beda `const` dan `let`? Kenapa `produk` pakai `const` tetapi isinya bisa berubah (`item.stok -= jumlah`)?	`const` tidak bisa di-assign ulang. Isi object atau array tetap bisa diubah, karena yang dikunci adalah variabelnya, bukan isinya.
Q2	Operator	Untuk apa operator `%` dipakai di proyek?	Sisa bagi, untuk pembulatan ke bawah kelipatan Rp100.
Q3	Ternary	Ubah satu `if/else` di kodemu menjadi ternary. Kapan sebaiknya tidak memakai ternary?	Ternary untuk memilih satu nilai. Jika bercabang banyak atau punya banyak langkah, `if/else` lebih terbaca.
Q4	Konversi	Kenapa memakai `Number(inputBayar.value)`?	`.value` selalu string. Tanpa konversi, `+` akan menggabungkan teks.
Q5	Function	Beda function declaration dan function expression?	Declaration kena hoisting (bisa dipanggil sebelum barisnya). Expression disimpan di variabel dan tidak bisa dipanggil sebelum dibuat.
Q6	Function	Kenapa `formatRupiah(angka = 0)` bisa dipanggil tanpa argumen?	Parameter default.
Q7	Return	Kenapa helper memakai `return`, bukan `console.log`?	`return` memberi nilai yang bisa dipakai lagi (dihitung, ditampilkan, diuji). `console.log` hanya menampilkan.

No	Topik	Pertanyaan	Kunci ringkas
Q8	First-class	Sebutkan tiga tempat function dikirim sebagai argumen di proyekmu.	`forEach`, `addEventListener`, `setTimeout`, `sort`, `filter`.
Q9	Closure	Jelaskan alur `daftarKupon.get("HEMAT10")(200000)`.	`get` mengambil function dari Map. Function itu (hasil `buatKupon(10)`) mengingat angka 10, lalu dipanggil dengan 200000 menghasilkan 20000.
Q10	Array	Kenapa `produk` berupa array berisi object, bukan beberapa array terpisah?	Satu produk memiliki banyak atribut yang berkaitan, dan object menjaganya tetap satu paket. Array untuk daftar yang berurutan.
Q11	Destructuring	Tulis ulang `const n = item.name; const h = item.price;` dengan destructuring.	`const { name: n, price: h } = item;` atau `const { name, price } = item;`
Q12	Array	Kenapa `Array.from(produk)` sebelum `sort`?	`sort` mengubah array aslinya. Salinan menjaga urutan data asli.
Q13	Map	Kenapa nota memakai `Map`, bukan object biasa?	Key `Map` boleh berupa object (produk). Key object biasa selalu diubah menjadi string.
Q14	Map	Kenapa `keranjang.has(salinanProduk)` bisa `false`?	Object berbeda alamat walaupun isinya sama.
Q15	DOM	Beda `textContent` dan `innerHTML`?	`textContent` menampilkan teks apa adanya. `innerHTML` memproses HTML, hanya untuk data tepercaya.
Q16	Event	Kenapa event dipasang setelah render?	Elemennya baru ada setelah `innerHTML` diisi, dan elemen lama (beserta event-nya) hilang setiap render ulang.
Q17	Event	Beda `addEventListener("input", renderKembalian)` dan `...renderKembalian()`?	Tanpa kurung: function dititipkan untuk dipanggil nanti. Dengan kurung: dipanggil sekarang dan hasilnya (undefined) yang dititipkan.
Q18	Modul	Beda named export dan default export? Contoh di proyekmu?	Named: banyak per file, di-import dengan `{ }` dan nama sama. Default: satu per file, bebas diberi nama. Contoh default: `keranjang`.
Q19	Modul	Kenapa `keranjang.js` tidak boleh menyentuh DOM?	Agar logikanya bisa diuji lewat Node dan dipakai ulang. Tampilan hanya urusan `main.js`.
Q20	CSS	Kenapa mode gelap cukup menimpa variabel?	Semua warna memakai `var(--...)`, sehingga nilai baru otomatis berlaku di semua aturan.
Q21	CSS	Kapan memakai flex dan kapan grid?	Flex untuk satu arah (baris atau kolom), grid untuk dua arah.
Q22	CSS	Untuk apa `position: relative` pada `.kartu-foto`?	Menjadi patokan posisi `absolute` anaknya (label harga, tombol ganti warna).

Bank tugas ubah-langsung (maksimal 2 menit)

No	Tugas	Lokasi perubahan	Catatan
U1	Ganti batas diskon 5% menjadi Rp100.000	`helper.js`, konstanta `BATAS_DISKON_SEDANG`	Papan info ikut berubah. Beberapa tes `hitungPersenDiskon` menjadi ❌, itu wajar karena aturan berubah. Minta siswa menjelaskan kenapa.
U2	Tambah kupon `BUAH20` (20%)	`helper.js`, `daftarKupon`: `["BUAH20", buatKupon(20)]`	Satu baris. Tidak perlu mengubah `main.js`.

No	Tugas	Lokasi perubahan	Catatan
U3	Batasi pembelian maksimal 5 kg per jenis	`keranjang.js`, `tambahItem`: tambah `|| jumlahSekarang >= 5` pada kondisi	Menunjukkan pemisahan logika.
U4	Ubah warna utama tombol menjadi biru	`style.css`, variabel `--aksi` di `:root` (dan `body.gelap` bila perlu)	Cukup satu tempat.
U5	Tambah produk "Semangka"	`data.js`, tambah object baru dengan id baru	Tampil otomatis. Boleh memakai gambar yang ada.
U6	Ganti teks sapaan malam	`helper.js`, `sapaByWaktu`	
U7	Tambah tingkat diskon 15% untuk subtotal >= Rp300.000	`helper.js`, `hitungPersenDiskon` (tambah satu ternary dan konstanta)	
U8	Ubah pembulatan menjadi kelipatan Rp500	`helper.js`, `hitungRingkasan`: `% 500`	
U9	Ubah batas riwayat menjadi 3 pesanan	`helper.js`, `MAKS_RIWAYAT`	Hanya jika siswa mengerjakan bonus riwayat.
U10	Ubah warna label harga	`style.css`, variabel `--pisang`	

D. Kunci jawaban pertanyaan di panduan siswa

Hanya yang jawabannya tidak langsung terlihat dari panduan.

Langkah	Pertanyaan	Kunci
1.1	Kenapa module harus lewat server?	Lihat M1.
1.2	Jika id `#tombol-bayar` diganti?	`querySelector` mengembalikan `null`. Saat `addEventListener` dipanggil muncul `TypeError`, sehingga seluruh `main.js` berhenti di baris itu.
1.3	Beda `justify-content` dan `align-items`?	`justify-content` mengatur sumbu utama (mendatar pada baris), `align-items` mengatur sumbu silang (menurun pada baris). Tanpa `sticky`, header ikut tergulung hilang.
1.4	Kenapa `minmax(0, 1fr)`?	`1fr` punya batas minimum otomatis sebesar isi terpanjang, sehingga isi lebar bisa mendorong kolom keluar layar. `minmax(0, 1fr)` membolehkan kolom menyusut.
1.5	`repeat(auto-fill, minmax(150px, 1fr))` dengan `gap: 12px` di layar 600 px dan 300 px?	600 px: 3 kolom (4 kolom butuh 4 x 150 + 3 x 12 = 636 px). 300 px: 1 kolom (2 kolom butuh 312 px). Jumlah kolom menyesuaikan lebar layar otomatis.
1.5	Kenapa `.kartu-foto` `position: relative`? Dari mana warna latar?	Sebagai patokan elemen `absolute` di dalamnya (label harga). Warna dari variabel `--warna` pada atribut `style` kartu.
1.6	`"18.000" * 2`?	`36`. JavaScript membaca `"18.000"` sebagai 18 (titik = desimal). Harga salah 1000 kali lipat tanpa ada error. Karena itu harga ditulis `18000`.
1.6	Kenapa `id` unik?	Dipakai untuk id elemen (`tambah-3`), sehingga `querySelector` menemukan tombol yang tepat.
2.1	`(32000).toLocaleString("id-ID")` dan `typeof`?	`"32.000"`, `"string"`.

Langkah	Pertanyaan	Kunci
2.1	Function expression vs declaration saat dipanggil sebelum barisnya?	Declaration bisa (hoisting). Expression: error karena variabel belum diisi.
2.3	Jika `let html;` tanpa nilai awal?	Hasilnya diawali teks `undefined`, karena `undefined + "<article..."` menjadi string.
3.1	`peta.has(a), peta.has(b), peta.size`	`true`, `false`, `1`.
3.2	Kenapa `(jumlah, item)`?	`Map.forEach` memberi `(value, key)`. Value-nya jumlah, key-nya object produk.
4.1	`3 * 5 / 100` dan `3 * (5 / 100)`? Kenapa kali dulu lebih aman?	`0.15` dan `0.15000000000000002`. Komputer menyimpan pecahan seperti `0.05` secara tidak tepat (biner). Dengan mengalikan dulu, hasil antara berupa bilangan bulat sehingga pembulatan galat berkurang. Pada harga kelipatan seribu di proyek ini kedua cara kebetulan sama, tetapi kebiasaan ini lebih aman untuk uang.
4.2	`73150 % 100`; `((a, f = () => 0) => f(a))(5)`	`50`; `0`.
4.3	Kenapa `.value` string walaupun `type="number"`?	`.value` pada elemen input selalu mengembalikan string.
4.3	`"input", renderKembalian` vs `renderKembalian()`	Lihat Q17.
4.4	Kenapa stok dikurangi setelah bayar?	Nota hanyalah rencana belanja. Stok baru berubah saat transaksi selesai. Jika dikurangi di nota, pembeli lain tidak bisa membeli meski belum dibayar, dan stok salah bila nota dihapus.
4.5	Kenapa `daftarKupon.get(...)(200000)` bisa dipanggil?	Value Map-nya adalah function.
5.1	Kenapa mode gelap cukup menimpa variabel?	Lihat Q20.
5.2 B1	`angka.sort()` dan `angka.sort((a, b) => a - b)`	`[1, 10, 100, 9]` dan `[1, 9, 10, 100]`.

E. Troubleshooting

Gejala	Penyebab umum	Perbaikan
Halaman polos, Console: `blocked by CORS policy`	Dibuka dengan klik dua kali	Buka lewat Live Server atau `python3 -m http.server 5500`
`Uncaught SyntaxError: ... does not provide an export named 'x'`	Salah ketik nama, lupa `export`, atau default dan named tertukar (`import x` vs `import { x }`)	Samakan nama di `export` dan `import`. Default tanpa kurung kurawal
`Cannot read properties of null (reading 'addEventListener')`	Id salah, atau elemen belum dirender	Periksa ejaan id. Pastikan `innerHTML` diisi sebelum `querySelector`
Tombol tidak bereaksi	Event dipasang sebelum render atau id tidak cocok (`tambah-1` vs `tambah1`)	`console.log` di handler untuk memastikan terpanggil. Cocokkan id di template dan di `querySelector`
Teks `undefined` di halaman	Function lupa `return` atau `let html;` tanpa `""`	Periksa baris `return` dan nilai awal
`NaN`	Operasi dengan string atau `undefined`	`console.log(typeof nilai)`. Pakai `Number()`

Gejala	Penyebab umum	Perbaikan
Total tidak sama dengan contoh	Data `data.js` berbeda dari data yang direkomendasikan	Cocokkan data, atau hitung ulang dengan data siswa
`node uji-logika.mjs`: `Cannot use import statement outside a module`	`package.json` hilang atau terminal tidak berada di folder proyek	`cd` ke folder proyek. Pastikan `"type": "module"` ada di `package.json`
`ERR_MODULE_NOT_FOUND`	Path import salah atau file belum dibuat	Periksa nama file dan `./` di depan path
Perubahan CSS tidak terlihat	Cache	`Ctrl + Shift + R`
CSS tidak berlaku pada elemen tertentu	Salah nama class, atau aturan lain lebih spesifik	DevTools, panel Styles: lihat aturan yang dicoret
Gambar tidak muncul	Path salah. Path relatif terhadap `index.html`	`assets/img/nama.svg`
Tombol Bayar tampak tak terjangkau di layar pendek	Nota lebih tinggi dari layar	Wajar. Gulir halaman. (Nota hanya menempel di layar tinggi)
Live Server tidak memuat ulang	Ekstensi tidak aktif	Klik "Go Live" di bilah bawah VS Code, atau muat ulang manual

---

F. Peta kode `contoh-jadi/`

Untuk guru yang ingin menunjukkan bagian tertentu saat mengajar.

File	Isi	Materi
`js/data.js`	`produk`: 6 object	Array of object, named export
`js/helper.js`	`sapaByWaktu` (declaration), `formatRupiah` (expression + default), `hitungPersenDiskon` dan `hitungDiskon` (arrow + ternary), `buatKupon` (closure), `daftarKupon` (Map berisi function), `hitungRingkasan` (default function, return object, `%`)	Function, first-class, Map
`js/keranjang.js`	`keranjang` (Map, default export), `tambahItem`, `kurangiItem`, `hapusItem`, `kosongkanKeranjang`, `hitungSubtotal`, `hitungJumlahItem`	Map dengan key object, `Map.forEach (value, key)`
`js/riwayat.js`	`riwayat` (array, default export), `catatPesanan` (push dan shift), `batalkanPesananTerakhir` (pop), `ambilRiwayatTerbaruDulu` (Array.from dan reverse)	Method array
`js/main.js`	Bagian 1 import (named, default, `import * as`). Bagian 2 `querySelector`. Bagian 4 `tampilkanToast` (`setTimeout` dengan callback). Bagian 6 `buatKartu`, `renderKatalog`. Bagian 7 `renderNota`, `renderKembalian`. Bagian 8 `renderRiwayat`. Bagian 9 event.	DOM, event, destructuring, template literal
`css/style.css`	Bagian 1 token (variabel, mode gelap). Bagian 5 grid katalog, kartu, label harga (`clip-path`). Bagian 6 nota (`conic-gradient`, `drop-shadow`, `grid-template-areas`). Bagian 8 media query	CSS native
`uji-logika.mjs`	35 tes: 25 WAJIB, 5 LEVEL 2, 5 BONUS	First-class function di dalam alat uji

Hal yang sengaja tidak dipakai: `for` loop, `map` dan `reduce` dalam kode siswa, `localStorage`, `fetch`, framework apa pun. Looping memakai `forEach` (sudah terlihat di `ESModule/index.mjs`). `filter` dan `sort` hanya muncul di fitur Bonus dan dibahas di M8.